

GamutX

Environment Variables

PYTHON SCRIPTING

Environment Variables

- What Are Environment Variables?
- Ways to View Environment Variables
- Common Environment Variables in Windows
- All Ways to View ENV Variable in Terminal

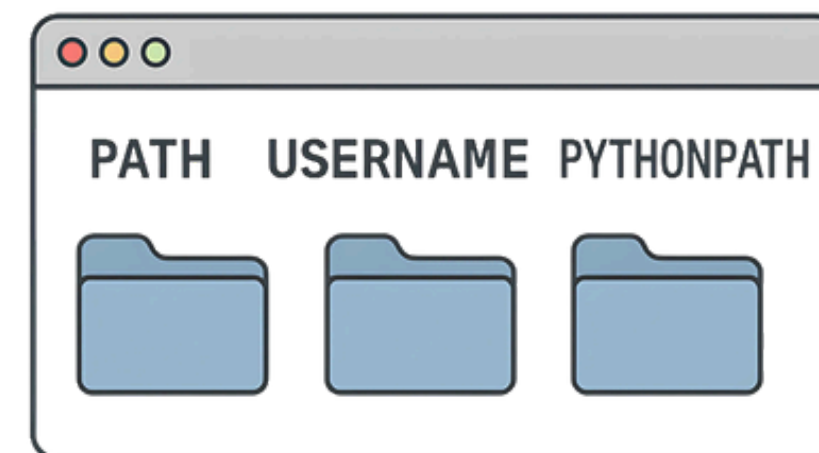
What Are Environment Variables?

- Environment variables are **key-value pairs**
- used by the **operating system** and **applications** to store **configuration data**.
- They act as a communication method between the system and software to define how programs should behave.

Think of them like labels
on jars in a kitchen



We can read the label and **grab the right ingredient**



instead of opening every jar to **find what we need**

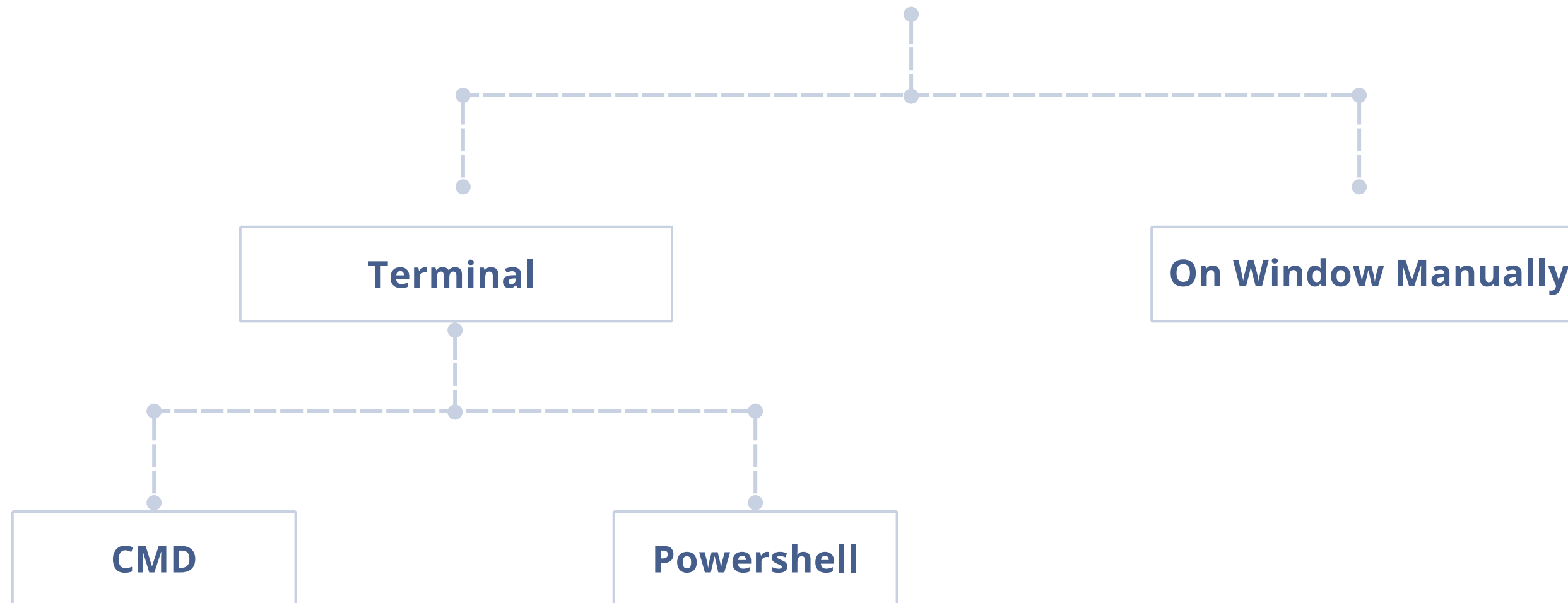
Variable

- **PATH**
- **PYTHONPATH**
- **MAYA_SCRIPT_PATH**

Purpose

- Tells the system where to look for executable programs
- Informs Python where to find additional modules
- Used by Autodesk Maya to locate user-defined MEL or Python scripts

Ways to View Environment Variables



Using terminal - View All ENV Variables

✓ Command Prompt (CMD)

Open CMD and Type → **set**

```
Microsoft Windows [Version 10.0.22631.5699]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>set
ACSetupSvcPort=23210
ACSvcPort=17532
ALLUSERSPROFILE=C:\ProgramData
APPDATA=C:\Users\pralhad\AppData\Roaming
CommonProgramFiles=C:\Program Files\Common Files
CommonProgramFiles(x86)=C:\Program Files (x86)\Common Files
CommonProgramW6432=C:\Program Files\Common Files
COMPUTERNAME=LAPTOP-H0K45GFP
ComSpec=C:\Windows\system32\cmd.exe
CUDA_PATH=C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9
CUDA_PATH_V12_9=C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9
DriverData=C:\Windows\System32\Drivers\DriverData
EFC_6248=1
EnableLog=INFO
HOMEDRIVE=C:
HOMEPATH=\Users\pralhad
IGCCSVC_DB=AQAAANCHnd8BFdERjHoAwE/Cl+sBAAAK0Uevf78ik+87v8N8LZPyAQAAAACAAAAAAQZgAAAAEAACAAAAD/ic9izdVH5iQQUfV51IB0oF1ZD
3NTI574Gg8yhqTZnAAAAA0gAAAAIAACAAAADLOCJmnRjuyFKwWHvw8p/Mn0Vp5+EsK05mKbKXvOMmYGAAAACl8GF0X1bejnz7HWbkzKiCeB32s6PhYUkkp
rMWkPjLDo+jh2QZ8Fi04Pl6Y09VD2eoC7vHdEEBC0B+qU00/J41Frj9vWMfBw+WSWbGkwcDZcFj651UzY+OJqbJXRmrgplAAAAAPsIhl7sR6seaaHiM3Gw68
kbNcfowqBwH+3Xs+P16p20aw8NssYjmvhWmMIkFBJaS+x7aDDWLzWhUicx2RuBARQ==
LOCALAPPDATA=C:\Users\pralhad\AppData\Local
LOGONSERVER=\\LAPTOP-H0K45GFP
NUMBER_OF_PROCESSORS=20
OS=Windows_NT
Path=C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9\bin;C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12
.9\libnvvp;C:\Program Files\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:
```

This shows all current environment variables.

✓ PowerShell

- Open PowerShell and type → `Get-ChildItem Env:`
- It will **lists all environment variables** in a table format.

```
Windows PowerShell
Install the latest Powershell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\pralhad> Get-ChildItem Env:

Name                           Value
----                           -
ACSetupSvcPort                 23210
ACSvcPort                     17532
ALLUSERSPROFILE                C:\ProgramData
APPDATA                       C:\Users\pralhad\AppData\Roaming
CommonProgramFiles             C:\Program Files\Common Files
CommonProgramFiles(x86)       C:\Program Files (x86)\Common Files
CommonProgramW6432            C:\Program Files\Common Files
COMPUTERNAME                  LAPTOP-H0K45GFP
ComSpec                       C:\Windows\system32\cmd.exe
CUDA_PATH                    C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9
CUDA_PATH_V12_9              C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9
DriverData                   C:\Windows\System32\Drivers\DriverData
EFC_6248                     1
EnableLog                    INFO
FPS_BROWSER_APP_PROFILE_STRING Internet Explorer
FPS_BROWSER_USER_PROFILE_STRING Default
HOMEDRIVE                     C:
HOMEPATH                     \Users\pralhad
IGCCSVC_DB                   AQAAANCMnd8BFdERjHoAwE/Cl+sBAAAAK0Uevf78ik+87v8N8lZPyAQAAAAACAAAAAAQZgAAAAEAACAAAD/i...
LOCALAPPDATA                  C:\Users\pralhad\AppData\Local
LOGONSERVER                   \\LAPTOP-H0K45GFP
NUMBER_OF_PROCESSORS          20
OS                            Windows_NT
Path                          C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9\bin;C:\Program Files\NVIDIA ...
```

✓ Linux

- Open Terminal and type → `env` or `printenv`

```
verlic@ubuntu: ~  
verlic@ubuntu:~$ env  
SHELL=/bin/bash  
SESSION_MANAGER=local/ubuntu:@/tmp/.ICE-unix/3257,unix/ubuntu:/tmp/.ICE-unix/3257  
QT_ACCESSIBILITY=1  
COLORTERM=truecolor  
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg  
XDG_MENU_PREFIX=gnome-  
GNOME_DESKTOP_SESSION_ID=this-is-deprecated  
GNOME_SHELL_SESSION_MODE=ubuntu  
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh  
MEMORY_PRESSURE_WRITE=c29tZSAyMDAwMDAgMjAwMDAwMAA=  
XMODIFIERS=@im=ibus  
DESKTOP_SESSION=ubuntu  
GTK_MODULES=gail:atk-bridge  
PWD=/home/verlic  
LOGNAME=verlic  
XDG_SESSION_DESKTOP=ubuntu  
XDG_SESSION_TYPE=wayland  
SYSTEMD_EXEC_PID=3301  
XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.BT6RA3  
HOME=/home/verlic  
USERNAME=verlic  
IM_CONFIG_PHASE=1
```

Keys

Values

✓ macos

- Open Terminal and type → `env` or `printenv`

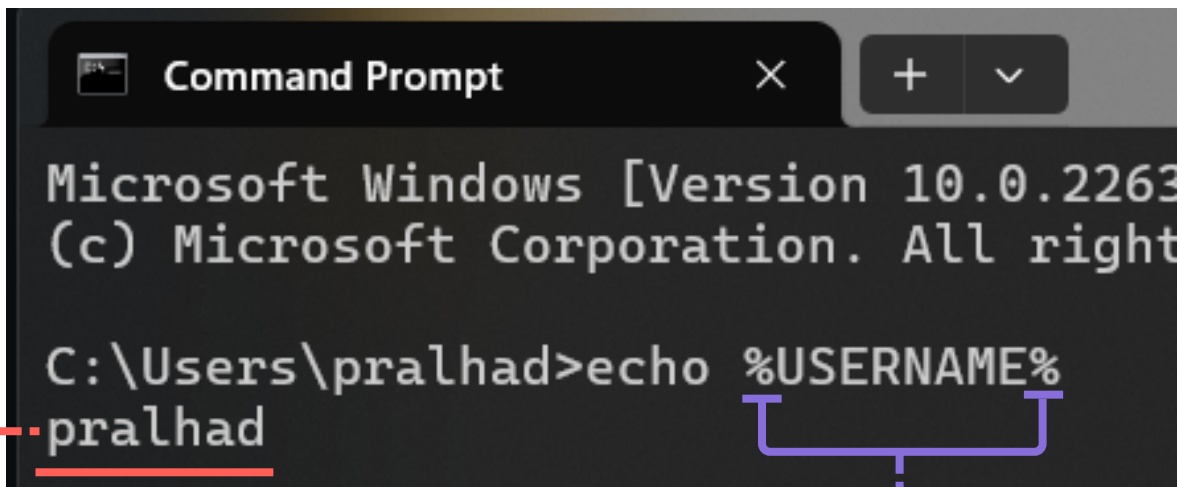
```
~ printenv
TERM_SESSION_ID=w0t0p0:490BA56C-C5BA-4218-A9A6-9504988A8638
SSH_AUTH_SOCK=/private/tmp/com.apple.launchd.BWGV7ERoQl/Listeners
LC_TERMINAL_VERSION=3.3.12
COLORFGBG=15;0
ITERM_PROFILE=Default
XPC_FLAGS=0x0
LANG=en_US.UTF-8
PWD=/Users/nphau
SHELL=/bin/zsh
TERM_PROGRAM_VERSION=3.3.12
TERM_PROGRAM=iTerm.app
PATH=/usr/local/opt/mysql@5.7/bin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:/usr/local/g
o/bin:/Library/Apple/usr/bin
LC_TERMINAL=iTerm2
COLORTERM=truecolor
TERM=xterm-256color
HOME=/Users/nphau
TMPDIR=/var/folders/x3/h4g6cqhn5rs6rs3bq0q7hf_00000gn/T/
USER=nphau
XPC_SERVICE_NAME=0
LOGNAME=nphau
ITERM_SESSION_ID=w0t0p0:490BA56C-C5BA-4218-A9A6-9504988A8638
__CF_USER_TEXT_ENCODING=0x0:0:0
SHLVL=1
```

Keys → Values

Using terminal - View Specific ENV Variables

On Windows

CMD



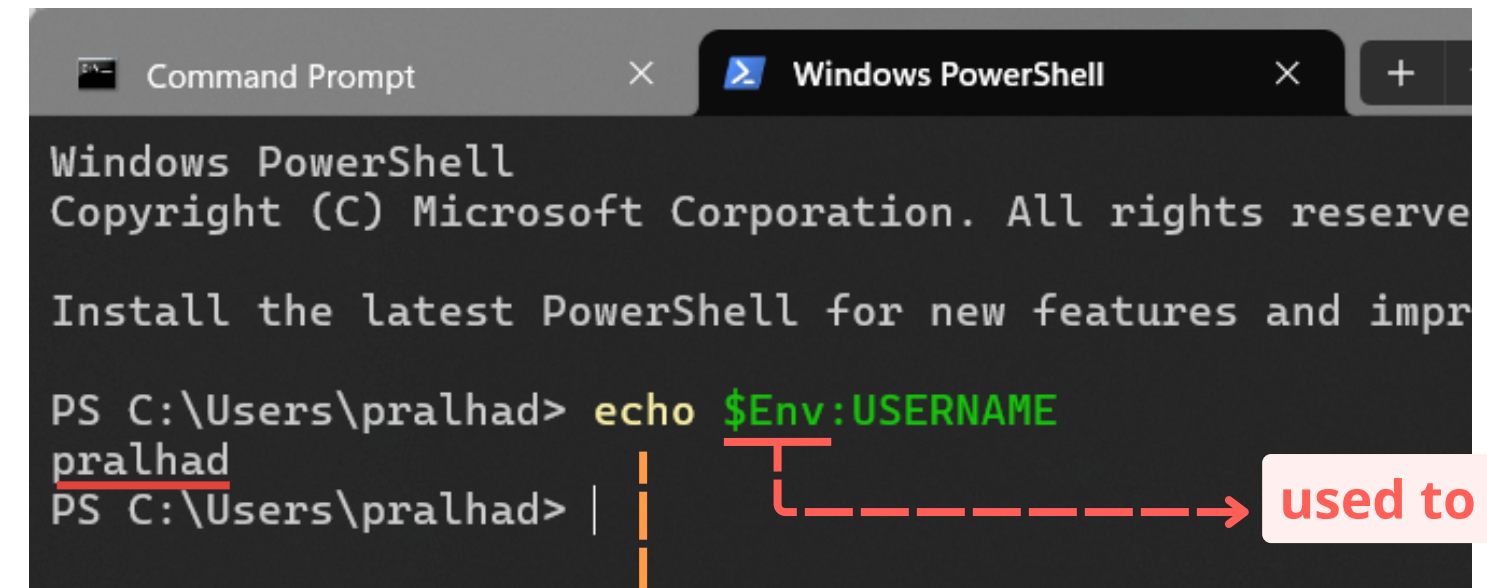
```
Microsoft Windows [Version 10.0.22631.1]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>echo %USERNAME%
pralhad
```

Current user name

used to access variable

PowerShell



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

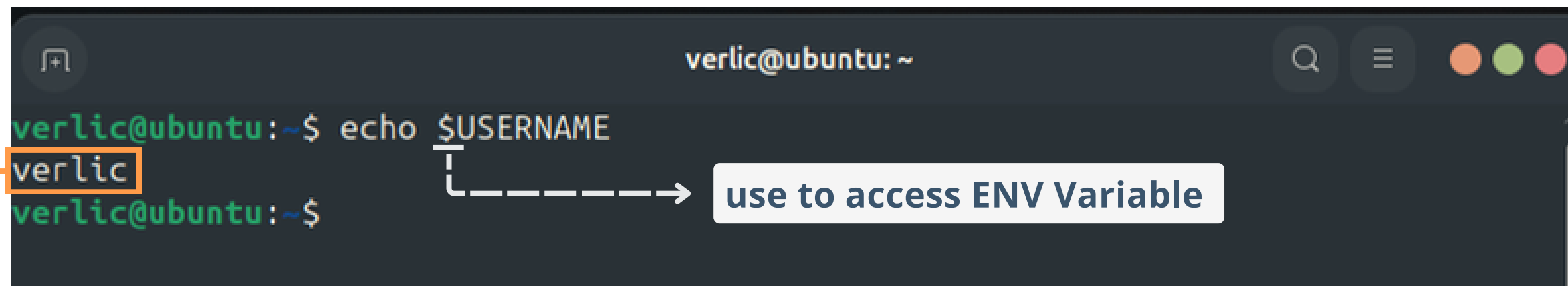
Install the latest PowerShell for new features and improvements!
https://aka.ms/PowerShellLatest

PS C:\Users\pralhad> echo $Env:USERNAME
pralhad
PS C:\Users\pralhad>
```

used to access variable

print output

On MacOS/Linux



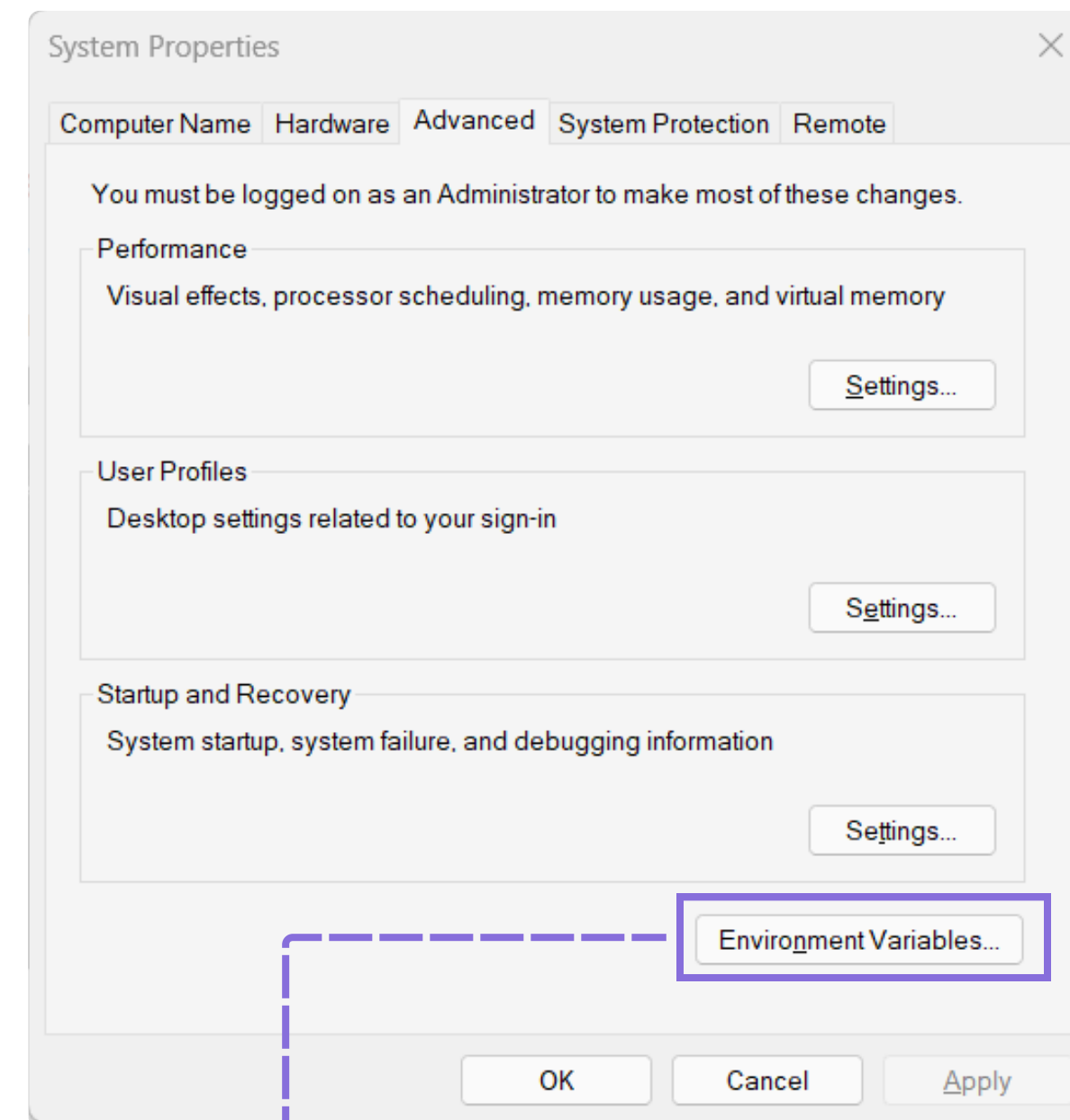
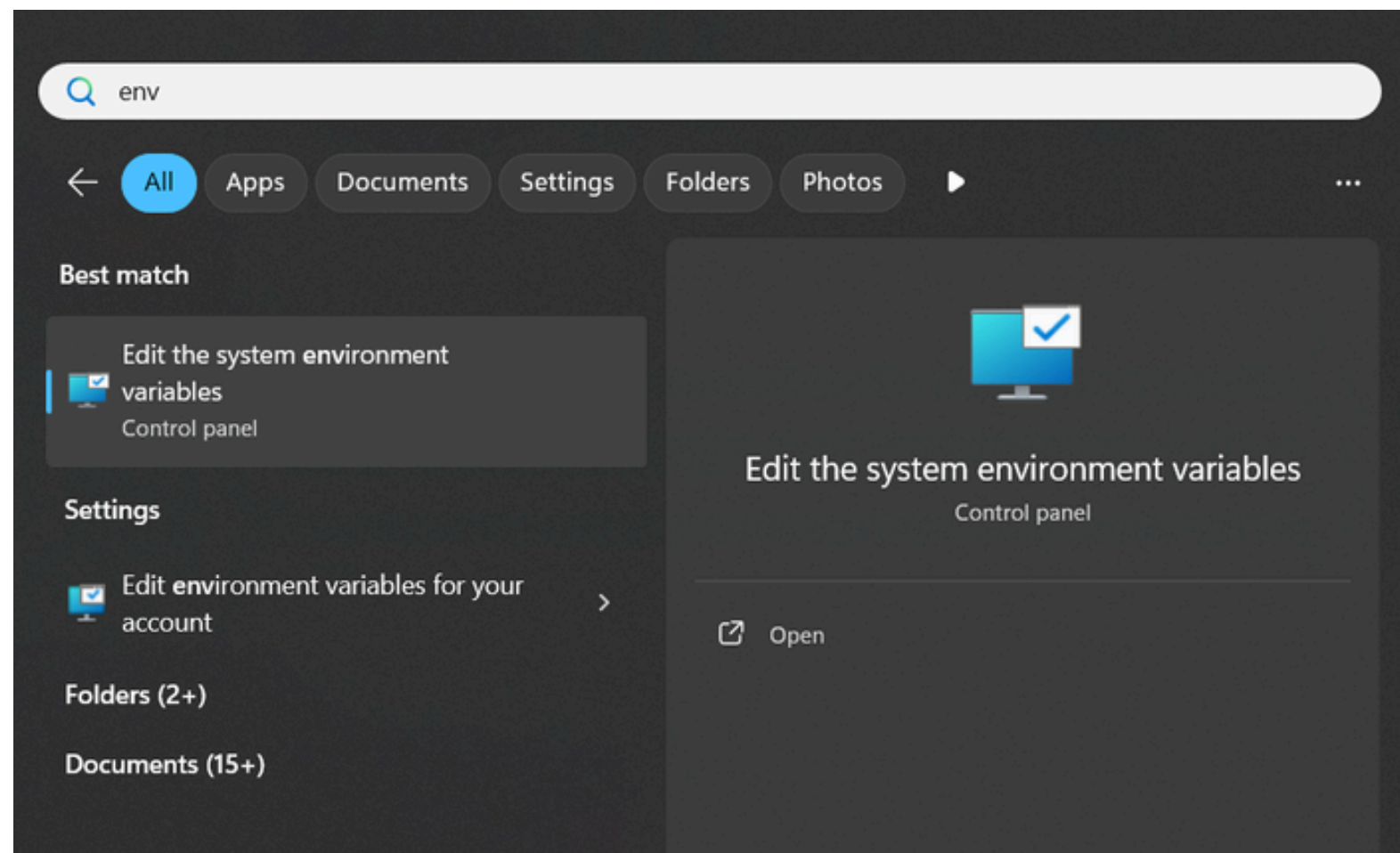
```
verlic@ubuntu: ~$ echo $USERNAME
verlic
verlic@ubuntu: ~$
```

use to access ENV Variable

Current user name

On Window Manually:

- Click the Windows **Start button**
- Type “**environment variables**” in the search

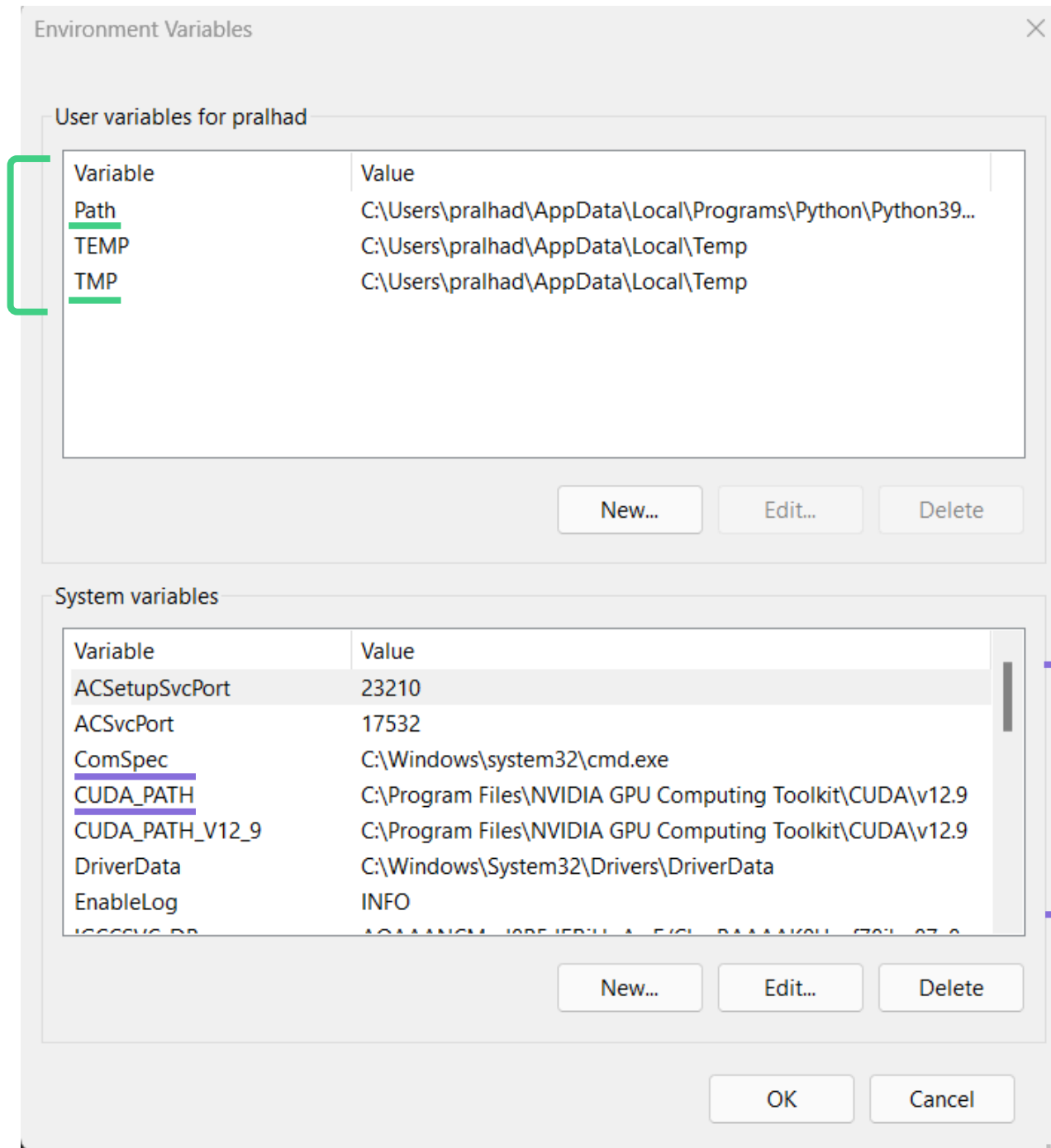


Click to **View** and edit **ENV Variable**



User Variables

These apply only to the current user.



System variables

These apply to all users on the machine.



Common Environment Variables in Windows

PATH

- Directories where executable **files are located**.
- **Example** → `C:\Windows\System32`

SYSTEMROOT

- Root directory of the **Windows** installation.
- **Example** → `C:\Windows`

APPDATA

- Path to application-**specific** data for the **user**.
- **Example** → `C:\Users\<User>\AppData\Roaming`

TEMP / TMP

- Directory for **temporary** files.
- **Example** → `C:\Users\<User>\AppData\Local\Temp`

USERNAME

- Current logged-in **user's name**.
- **Example** → `JohnDoe`

USERPROFILE

- Path to the **user's home folder**.
- **Example** → `C:\Users\JohnDoe`

LOCALAPPDATA

- Local (**non-roaming**) application data.
- **Example** → `C:\Users\<User>\AppData\Local`

All Ways to View ENV Variable in Terminal

macOS / Linux

- View all environment variables

```
env  
# or  
printenv
```

- View a specific variable

```
echo $PATH  
echo $HOME
```

Windows

- View all environment variables

Command Prompt (CMD)

```
set
```

PowerShell

```
Get-ChildItem Env:
```

- View a specific variable

CMD

```
echo %USERNAME%
```

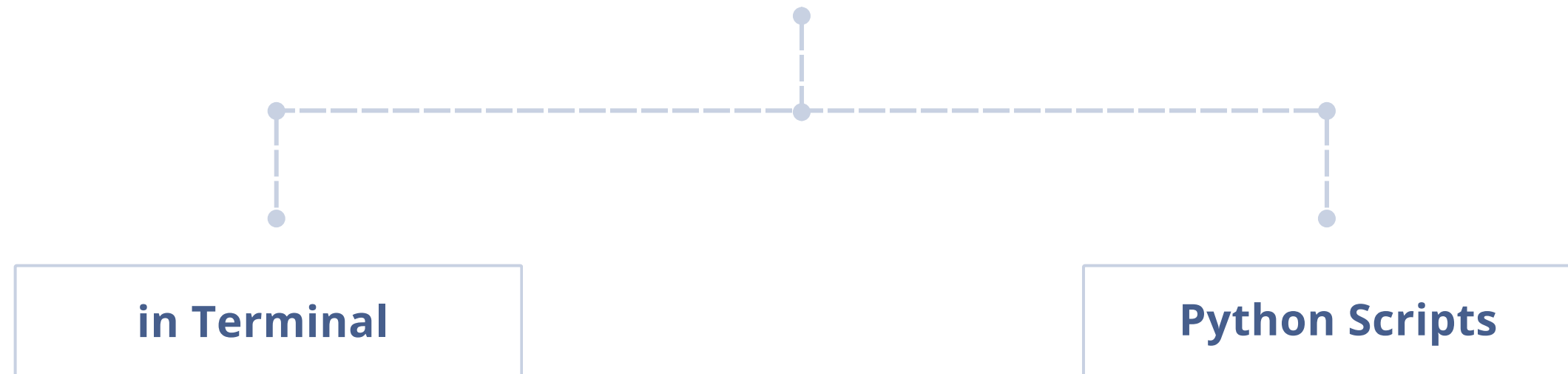
PowerShell

```
echo $Env:USERNAME
```

Ways to Check ENV Variables using Python

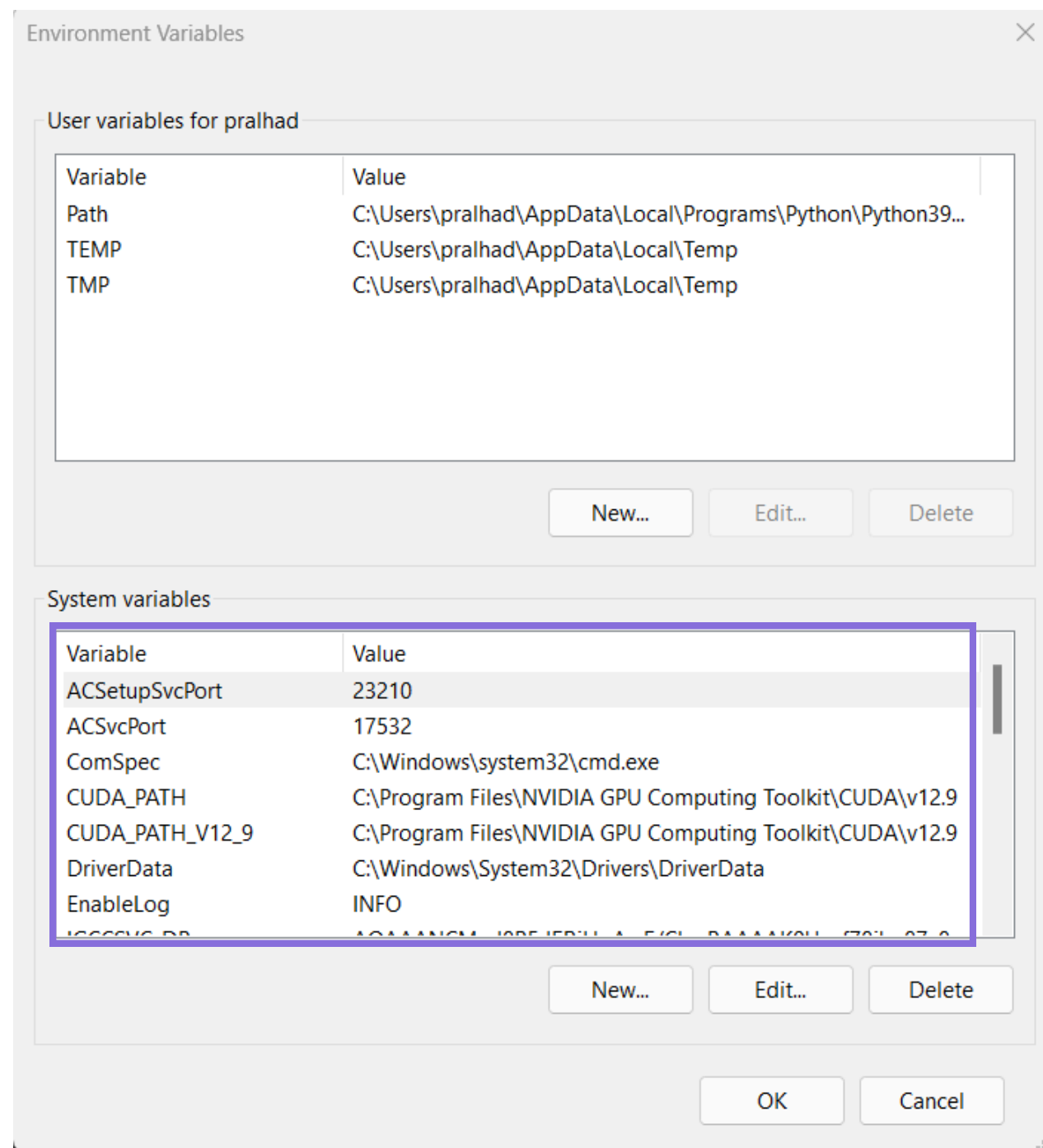
- Check ENV Variable in Terminal Using Python
- Check ENV Variable Using Python Script

Ways to “CHECK” Environment Variables using Python

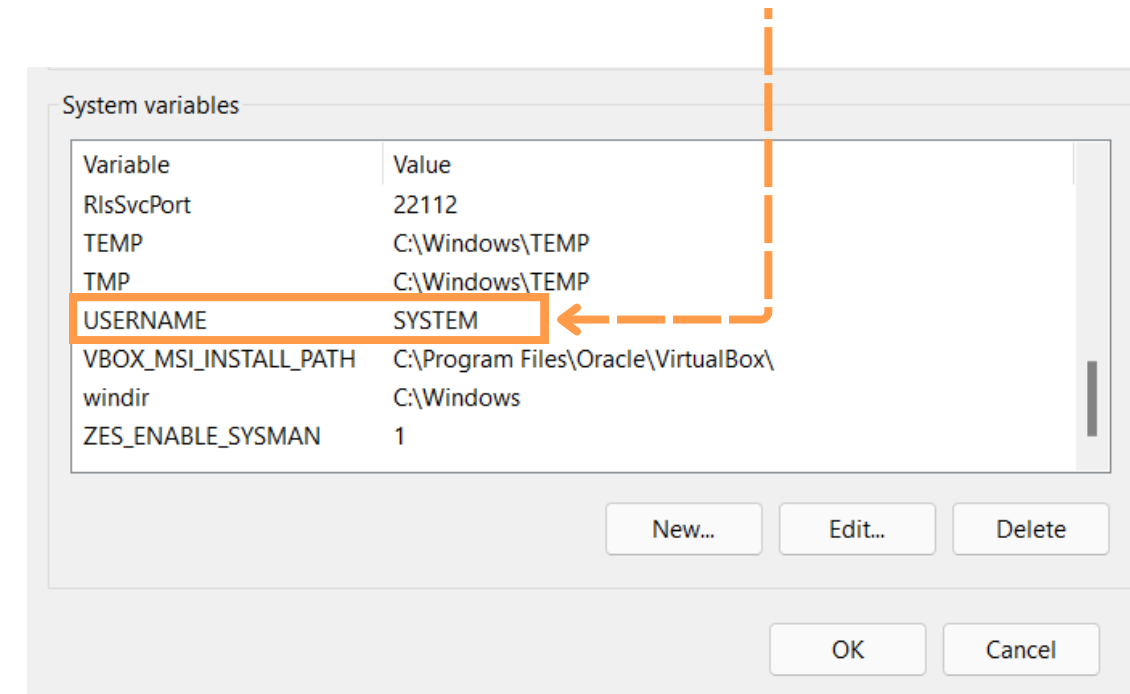


Check ENV Variable in Terminal - Using Python

Check All Environment Variables



Check a Specific Environment Variable



Check All Environment Variables

- Open a terminal (Command Prompt, PowerShell, or macOS/Linux Terminal).

```
Command Prompt - python
C:\Users\pralhad>python
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> print(os.environ)
environ({'ACSETUPSVCPOR': '23210', 'ACSVCPOR': '17532', 'ALLUSERSPROFILE': 'C:\\\\ProgramData', 'APPDATA': 'C:\\\\Users\\p
ralhad\\AppData\\Roaming', 'COMMONPROGRAMFILES': 'C:\\\\Program Files\\Common Files', 'COMMONPROGRAMFILES(X86)': 'C:\\\\Prog
ram Files (x86)\\Common Files', 'COMMONPROGRAMW6432': 'C:\\\\Program Files\\Common Files', 'COMPUTERNAME': 'LAPTOP-H0K45GF
P', 'COMSPEC': 'C:\\\\Windows\\system32\\cmd.exe', 'CUDA_PATH': 'C:\\\\Program Files\\NVIDIA GPU Computing Toolkit\\CUDA\\v1
2.9', 'CUDA_PATH_V12_9': 'C:\\\\Program Files\\NVIDIA GPU Computing Toolkit\\CUDA\\v12.9', 'DRIVERDATA': 'C:\\\\Windows\\Sys
tem32\\Drivers\\DriverData', 'EFC_16972': '1', 'ENABLELOG': 'INFO', 'FPS_BROWSER_APP_PROFILE_STRING': 'Internet Explorer
', 'FPS_BROWSER_USER_PROFILE_STRING': 'Default', 'HOMEDRIVE': 'C:', 'HOMEPATH': '\\\\Users\\pralhad', 'IGCCSVC_DB': 'AQAAA
NCMnd8BFdERjHoAwE/C1+sBAAAAK0Uevf78ik+87v8N8LZPyAQAAAACAAAAAAQZgAAAAEAACAAAAD/ic9izdVH5iQQUfV51IBOoF1ZD3NTI574Gg8yhqTZn
AAAAAAOgAAAAIAACAAAADLOCJmnrJuyFKwWHvw8p/Mn0Vp5+EsKO5mKbKXvOMmYGAAAACl8GF0X1bejnz7HWbkzKiCeB32s6PhYUkkprMWkPjLDo+jh2QZ8
Fi04Pl6YO9VD2eoC7vHdEEBC0B+qU00/J41Frj9vWMfBw+WSWbGkwcDZcFj651UzY+OJqbJXRmrgplAAAAAPsIhl7sR6seaaHiM3Gw68kbNcfowqBwH+3Xs+
P16p20aw8NssYjmvhWmMIkFBJaS+x7aDDWLzWhUicx2RuBARQ==', 'LOCALAPPDATA': 'C:\\\\Users\\pralhad\\AppData\\Local', 'LOGONSERVER
': '\\\\LAPTOP-H0K45GFP', 'MAYA_PLUG_IN_PATH': 'Z:\\\\StudioTools\\CustomPlugin', 'NUMBER_OF_PROCESSORS': '20', 'OS': 'Win
dows_NT', 'PATH': 'C:\\\\Program Files\\NVIDIA GPU Computing Toolkit\\CUDA\\v12.9\\bin;C:\\\\Program Files\\NVIDIA GPU Compu
ting Toolkit\\CUDA\\v12.9\\libnvvp;C:\\\\Program Files\\Common Files\\Oracle\\Java\\javapath;C:\\\\Windows\\system32;C:\\\\Win
dows;C:\\\\Windows\\System32\\Wbem;C:\\\\Windows\\System32\\WindowsPowerShell\\v1.0\\;C:\\\\Windows\\System32\\OpenSSH\\;C:\\\\P
rogram Files (x86)\\NVIDIA Corporation\\PhysX\\Common;C:\\\\Program Files\\NVIDIA Corporation\\NVIDIA NvDLISR;C:\\\\Program
Files\\dotnet\\;C:\\\\ffmpeg\\bin;C:\\\\Program Files\\Blender Foundation\\Blender 4.4\\;C:\\\\Program Files\\Git\\cmd;C:\\\\Pro
gram Files\\nodejs\\;c:\\\\Users\\pralhad\\AppData\\Local\\Programs\\Trae\\bin;C:\\\\Users\\pralhad\\AppData\\Local\\Program
ata\\Local\\Programs\\Microsoft VS Code\\bin;C:\\\\Users\\pralhad\\AppData\\Local\\Programs\\Ollama;C:\\\\Users\\pralhad\\Ap
pData\\Roaming\\npm;C:\\\\Users\\pralhad\\AppData\\Local\\Programs\\Windsurf\\bin;C:\\\\Users\\pralhad\\AppData\\Local\\Prog
rams\\cursor\\resources\\app\\bin;C:\\\\Users\\pralhad\\Desktop\\file_folder_generator - Copy;C:\\\\Users\\pralhad\\Desktop\\
file_folder_generator;C:\\\\Program Files\\NVIDIA Corporation\\Nsight Compute 2025.2.1\\;C:\\\\Program Files\\NVIDIA Corpor
ation\\NVIDIA App\\NvDLISR;C:\\\\Users\\pralhad\\AppData\\Roaming\\Python\\Scripts;C:\\\\Users\\pralhad\\Desktop\\MyTools;C:
\\\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\Scripts\\;C:\\\\Users\\pralhad\\AppData\\Local\\Programs\\Py
```

Check a Specific Environment Variable

- In CMD or PowerShell run →

```
Command Prompt - python
Microsoft Windows [Version 10.0.22631.5699]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>python
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> print(os.environ.get('USERNAME'))
pralhad
>>> print(os.environ.get('USERPROFILE'))
C:\Users\pralhad
>>> |
```

→ Current logged-in **user's name**.

You can also check, like '**USERNAME**', '**TEMP**', or '**JAVA_HOME**'.

Verifying Output

```
Command Prompt - python
C:\Users\pralhad>python
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> print(os.environ)
environ({'ACSETUPSVCPORT': '23210', 'ACSVCPORT': '17532', 'ALLUSERSPROFILE': 'C:\\ProgramData', 'APPDATA': 'C:\\Users\\p
ralhad\\AppData\\Roaming', 'COMMONPROGRAMFILES': 'C:\\Program Files\\Common Files', 'COMMONPROGRAMFILES(X86)': 'C:\\Prog
ram Files (x86)\\Common Files', 'COMMONPROGRAMW6432': 'C:\\Program Files\\Common Files', 'COMPUTERNAME': 'LAPTOP-H0K45GF
P', 'COMSPEC': 'C:\\Windows\\system32\\cmd.exe', 'CUDA_PATH': 'C:\\Program Files\\NVIDIA GPU Computing Toolkit\\CUDA\\v1
2.9', 'CUDA_PATH_V12_9': 'C:\\Program Files\\NVIDIA GPU Computing Toolkit\\CUDA\\v12.9', 'DRIVERDATA': 'C:\\Windows\\Sys
tem32\\Drivers\\DriverData', 'EFC_16972': '1', 'ENABLELOG': 'INFO', 'FPS_BROWSER_APP_PROFILE_STRING': 'Internet Explorer
', 'FPS_BROWSER_USER_PROFILE_STRING': 'Default', 'HOMEDRIVE': 'C:', 'HOMEPATH': '\\\\Users\\pralhad', 'IGCCSVC_DB': 'AQAAA
NCMnd8BFdERjHoAwE/CL+sBAAAAK0Uevf78ik+87v8N8lZPyAQAAAAACAAAAAAQZgAAAAEAACAAAAD/ic9izdVH5iQQUfV51IB0oF1ZD3NTI574Gg8yhqTZn
AAAAAAGAAAAAIAACAAAADLOCJmnRjuyFKwWHvw8p/Mn0Vp5+EsK05mKbKXvOMmYGAAAACl8GF0X1bejnz7HWbkzKiCeB32s6PhYUkkprMWkPjLDo+jh2QZ8
Fi04Pl6Y09VD2eoC7vHdEEBC0B+qU00/J41Frj9vWMfBw+WSWbGkwcDZcFj651UzY+OJqbJXRmrgplAAAAAPsIhl7sR6seaaHiM3Gw68kbNcfowqBwH+3Xs+
P16p20aw8NssYjmvhWmMIkFBJaS+x7aDDWLzWhUicx2RuBARQ==', 'LOCALAPPDATA': 'C:\\Users\\pralhad\\AppData\\Local', 'LOGONSERVER
': '\\\\LAPTOP-H0K45GFP', 'MAYA_PLUG_IN_PATH': 'Z:\\StudioTools\\CustomPlugin', 'NUMBER_OF_PROCESSORS': '20', 'OS': 'Win
dows_NT', 'PATH': 'C:\\Program Files\\NVIDIA GPU Computing Toolkit\\CUDA\\v12.9\\bin;C:\\Program Files\\NVIDIA GPU Compu
ting Toolkit\\CUDA\\v12.9\\libnvvp;C:\\Program Files\\Common Files\\Oracle\\Java\\javapath;C:\\Windows\\system32;C:\\Win
dows;C:\\Windows\\System32\\Wbem;C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\;C:\\Windows\\System32\\OpenSSH\\;C:\\P
rogram Files (x86)\\NVIDIA Corporation\\PhysX\\Common;C:\\Program Files\\NVIDIA Corporation\\NVIDIA NvDLISR;C:\\Program
Files\\dotnet\\;C:\\ffmpeg\\bin;C:\\Program Files\\Blender Foundation\\Blender 4.4\\;C:\\Program Files\\Git\\cmd;C:\\Pro
gram Files\\nodejs\\;c:\\Users\\pralhad\\AppData\\Local\\Programs\\Trae\\bin;C:\\Users\\pralhad\\AppData\\Local\\Program
ata\\Local\\Programs\\Microsoft VS Code\\bin;C:\\Users\\pralhad\\AppData\\Local\\Programs\\Ollama;C:\\Users\\pralhad\\Ap
pData\\Roaming\\npm;C:\\Users\\pralhad\\AppData\\Local\\Programs\\Windsurf\\bin;C:\\Users\\pralhad\\AppData\\Local\\Prog
rams\\cursor\\resources\\app\\bin;C:\\Users\\pralhad\\Desktop\\file_folder_generator - Copy;C:\\Users\\pralhad\\Desktop\\
\\file_folder_generator;C:\\Program Files\\NVIDIA Corporation\\Nsight Compute 2025.2.1\\;C:\\Program Files\\NVIDIA Corpor
ation\\NVIDIA App\\NvDLISR;C:\\Users\\pralhad\\AppData\\Roaming\\Python\\Scripts;C:\\Users\\pralhad\\Desktop\\MyTools;C:
\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\Scripts\\;C:\\Users\\pralhad\\AppData\\Local\\Programs\\Py
```

Environment Variables

User variables for pralhad

Variable	Value
MAYA_PLUG_IN_PATH	Z:\StudioTools\CustomPlugin
Path	C:\Users\pralhad\AppData\Local\Programs\Python\Python39...
TEMP	C:\Users\pralhad\AppData\Local\Temp
TMP	C:\Users\pralhad\AppData\Local\Temp

New... Edit... Delete

System variables

Variable	Value
ACSetupSvcPort	23210
ACSvcPort	17532
ComSpec	C:\Windows\system32\cmd.exe
CUDA_PATH	C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9
CUDA_PATH_V12_9	C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9
DriverData	C:\Windows\System32\Drivers\DriverData
EnableLog	INFO
IGCCSVC_DB	AQAAA...

New... Edit... Delete

OK Cancel

Using os.environ

dictionary-like object

stores all environment variables as key-value pairs.

```
PS C:\Users\pralhad\Desktop\New folder> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39/python.exe "c:/Users/pralhad/Desktop/New folder/c.py"
environ({'ACSETUPVCPORT': '23210', 'ACSVCPORT': '17532', 'ALLUSERSPROFILE': 'C:\\ProgramData', 'APPDATA': 'C:\\Users\\pralhad\\AppData\\Roaming', 'CHROME_CRASHPAD_PIPE_NAME': '\\\\.\\pipe\\crashpad_12508_ZPVBLSBGYQWLKRH', 'COMMONPROGRAMFILES': 'C:\\Program Files\\Common Files', 'COMMONPROGRAMFILES(X86)': 'C:\\Program Files (x86)\\Common Files', 'COMMONPROGRAMW6432': 'C:\\Program Files\\Common Files', 'COMPUTERNAME': 'LAPTOP-H0K45GFP', 'COMSPEC': 'C:\\Windows\\system32\\cmd.exe', 'CUDA_PATH': 'C:\\Program Files\\NVIDIA GPU Computing Toolkit\\CUDA\\v12.9', 'CUDA_PATH_V12_9': 'C:\\Program Files\\NVIDIA GPU Computing Toolkit\\CUDA\\v12.9', 'DRIVERDATA': 'C:\\Windows\\System32\\Drivers\\DriverData', 'EFC_6248': '1', 'ENABLELOG': 'INFO', 'FPS_BROWSER_APP_PROFILE_STRING': 'Internet Explorer', 'FPS_BROWSER_USER_PROFILE_STRING': 'Default', 'HOMEDRIVE': 'C:', 'HOMEPATH': '\\\\Users\\pralhad', 'IGCCSVC_DB': 'AQAAWNCmd8BFderjHoAwE/C1+sBAAAAK0Uevf78ik+87v8N81ZPyAQAAAACAAAAAAAAAQA2gAAAEAAACAAAD/ic9izdVH5IQQUFVS1IBOoF1ZD3NTI574GgSyhqTZnAAAAAAGAAAAIAACAAADLOCJmRjuyFKwHnw8p/PnOVp5+EskOSmkBXvOMMyGAAACl8GF0X1bejnz7HwbkzKiCeB32s6PhyUkkprMkpJldo+jh2QZ8Fi04Pl6YO9VD2eoC7VhdEEBCOB+qUR0/341Frj9WMFBw+WSMbGlowCDZcfj651UZy+OJqbIXBrngplAAAAAPsTh17sR6seaaHiM3Gw68kbncfowqBw++3Xs+P16p20aw8NssYjmvhwMIkFBjaS+x7aDDMLzhUicx2RuBARQ==', 'LOCALAPPDATA': 'C:\\Users\\pralhad\\AppData\\Local', 'LOGONSERVER': '\\\\LAPTOP-H0K45GFP', 'NUMBER_OF_PROCESSORS': '20', 'ORIGINAL_XDG_CURRENT_DESKTOP': 'undefined', 'OS': 'Windows_NT', 'PATH': 'C:\\Program Files\\NVIDIA GPU Computing Toolkit\\CUDA\\v12.9\\bin;C:\\Program Files\\NVIDIA GPU Computing Toolkit\\CUDA\\v12.9\\libnvvp;C:\\Program Files\\Common Files\\Oracle\\Java\\javapath;C:\\Windows\\system32;C:\\Windows;C:\\Windows\\System32\\Wbem;C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\;C:\\Windows\\System32\\OpenSSH\\;C:\\Program Files (x86)\\NVIDIA Corporation\\PhysX\\Common;C:\\Program Files\\NVIDIA Corporation\\NVIDIA nvdlsr;C:\\Program Files\\dotnet\\;C:\\ffmpeg\\bin;C:\\Program Files\\Blender Foundation\\Blender 4.4\\;C:\\Program Files\\Git\\cmd;C:\\Program Files\\nodejs\\;c:\\Users\\pralhad\\AppData\\Local\\Programs\\Trae\\bin;C:\\Users\\pralhad\\AppData\\AppData\\Local\\Programs\\Python\\Python39\\;C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python311\\Scripts\\;C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python311\\;C:\\Users\\pralhad\\scoop\\shims;C:\\Windows\\system32;C:\\Windows;C:\\Windows\\System32\\Wbem;C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\;C:\\Windows\\System32\\OpenSSH\\;C:\\Program Files (x86)\\NVIDIA Corporation\\PhysX\\Common;C:\\Program Files\\NVIDIA Corporation\\AppData\\Local\\Programs\\Python\\Python39\\;C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python311\\Scripts\\;C:\\Users\\AppData\\Local\\Programs\\Python\\Python39\\;C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python311\\Scripts\\;C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\AppData\\Local\\Programs\\Python\\Python39\\;C:\\
```

```
path_var = os.getenv("PATH")
print(path_var)
```

- ## Provide name of Current User

- Gives a List of **Executable Program Paths** → like **python, git, ffmpeg, etc.**

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AUGMENT NEXT EDIT Python + v [ ] [ ] ... [ ] [X]
● PS C:\Users\pralhad\Desktop\New folder> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39/python.exe "c:/Users/pralhad/Desktop/New folder/c.py"
○ pralhad
C:\Program Files\WIDIA GPU Computing Toolkit\CUDA\v12.9\bin;C:\Program Files\WIDIA GPU Computing Toolkit\CUDA\v12.9\libnvvp;C:\Program Files\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files (x86)\WIDIA Corporation\PhysX\Common;C:\Program Files\WIDIA Corporation\WIDIA NvDLISR;C:\Program Files\dotnet\;C:\ffmpeg\bin;C:\Program Files\Blender Foundation\Blender 4.4\;C:\Program Files\Git\cmd;C:\Program Files\nodejs\;c:\Users\pralhad\AppData\Local\Programs\Trae\bin;C:\Users\pralhad\AppData\Local\Programs\Python\Python39\Scripts\;C:\Users\pralhad\AppData\Local\Programs\Python\Python39\;C:\Users\pralhad\AppData\Local\Programs\Microsoft VS Code\bin;C:\Users\pralhad\AppData\Local\Programs\Ollama;C:\Users\pralhad\AppData\Roaming\npm;C:\Users\pralhad\AppData\Local\Programs\Winsurf\bin;C:\Users\pralhad\AppData\Local\Programs\cursor\resources\app\bin;C:\Users\pralhad\Desktop\file_folder_generator - Copy;C:\Users\pralhad\Desktop\file_folder_generator;C:\Program Files\WIDIA Corporation\Wight Compute 2025.2.1\;C:\Program Files\WIDIA Corporation\WIDIA App\WOLISR;C:\Users\pralhad\AppData\Roaming\Python\Scripts;C:\Users\pralhad\AppData\Local\Programs\Python\Python39\Scripts\;C:\Users\pralhad\AppData\Local\Programs\Python\Python39\;C:\Users\pralhad\AppData\Local\Programs\Python\Python311\Scripts\;C:\Users\pralhad\AppData\Local\Programs\Python\Python311\;C:\Users\pralhad\scoop\shims;C:\Windows\system32;C:\Windows;C:\Windows\System32\wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files (x86)\WIDIA Corporation\PhysX\Common;C:\Program Files\WIDIA Corporation\WIDIA NvDLISR;C:\Program Files\dotnet\;C:\ffmpeg\bin;C:\Program Files\Git\cmd;C:\Program Files\nodejs\;C:\Program Files\Blender Foundation\Blender 4.4\;C:\Windows\system32;C:\Windows;C:\Windows\System32\wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files (x86)\WIDIA Corporation\PhysX\Common;C:\Program Files\WIDIA Corporation\WIDIA NvDLISR;C:\Program Files\dotnet\;C:\ffmpeg\bin;C:\Program Files\Git\cmd;C:\Program Files\nodejs\;C:\Program Files\Blender Foundation\Blender 4.4\;C:\Windows\system32;C:\Windows;C:\Windows\System32\wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files (x86)\WIDIA Corporation\PhysX\Common;C:\Program Files\WIDIA Corporation\WIDIA NvDLISR;C:\Program Files\dotnet\;C:\ffmpeg\bin;C:\Users\pralhad\AppData\Local\Programs\Microsoft VS Code\bin;C:\Users\pralhad\AppData\Local\Programs\Ollama;C:\Users\pralhad\AppData\Roaming\npm;C:\Users\pralhad\AppData\Local\Programs\Winsurf\bin;C:\Users\pralhad\.dotnet\tools;c:\Users\pralhad\.vscode\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\scripts\noConfigScripts;c:\Users\pralhad\AppData\Roaming\Code\User\globalStorage\github.copilot-chat\debugCommand
PS C:\Users\pralhad\Desktop\New folder>

```


Understanding the PATH Variable

- What Is the PATH Environment Variable?
- How Does PATH Work?
- What If PATH for Python Is Not Set?
- Why Is the PATH Variable Important?
- Viewing the PATH Variable in the Terminal
- Modifying the PATH Variable
- Common Issues While Getting PATH

What is the PATH Environment Variable?

- The **PATH** environment variable tells computer **where to look for executable files**
- when we type a command like **python**, **git**, or **maya** in the **terminal**.

Key Points

- It's a **list of folder paths**
- When you run a command, the system searches these folders **in order**
- The first matching executable it finds is the one it runs

Platform Differences

OS	Separator Between Paths	Example
• Windows	→ Semicolon ;	→ <code>C:\Python39\;C:\Windows\System32\</code>
• macOS/Linux	→ Colon :	→ <code>/usr/local/bin:/usr/bin:/bin</code>

How Does PATH Work?

Let's say you type "python" in terminal

```
C:\Users\pralhad>python
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

Here's what happens:

- The system looks through **all** directories listed in the **PATH** variable.
- It checks each one to see if it **contains** an **executable** called **python**.
- If **found**, it runs it.

What If PATH for Python Is Not Set?

Demo:

- Let's **Remove** Python's directory from the **PATH** variable.
- Run python again

The image shows two side-by-side screenshots of Windows system settings. The left screenshot is the 'System variables' window, with the 'Path' variable selected and highlighted by a purple box. A dashed purple arrow points from this box to a callout that says 'Double click to see all save paths'. The right screenshot is the 'Edit environment variable' window, showing a list of paths. The Python path 'C:\Users\pralhad\AppData\Local\Programs\Python\Python39\' is highlighted by an orange box, with a dashed orange arrow pointing to a callout that says 'Select Python path'. The 'Delete' button on the right is highlighted by a green box, with a dashed green arrow pointing to a callout that says 'click to remove path'. The 'OK' button at the bottom is highlighted by a red box, with a dashed red arrow pointing to a callout that says 'Click to save changes'.

System variables

Variable	Value
EnableLog	INFO
IGCCSVC_DB	AQAAANCMnd8BFdERjHoAwE/CI+sBAAAAK0Uevf78ik+87v8...
NUMBER_OF_PROCESSORS	20
OS	Windows NT
Path	C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9...
PATHEXT	.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTU...	AMD64

New... Edit... Delete

OK Cancel

Double click to see all save paths

Edit environment variable

C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common
C:\Program Files\NVIDIA Corporation\NVIDIA NvDLISR
C:\Program Files\dotnet\
C:\ffmpeg\bin
C:\Program Files\Blender Foundation\Blender 4.4\
C:\Program Files\Git\cmd
C:\Program Files\nodejs\
c:\Users\pralhad\AppData\Local\Programs\Trae\bin
C:\Users\pralhad\AppData\Local\Programs\Microsoft VS Co...
C:\Users\pralhad\AppData\Local\Programs\Ollama
C:\Users\pralhad\AppData\Roaming\npm
C:\Users\pralhad\AppData\Local\Programs\Windsurf\bin
C:\Users\pralhad\AppData\Local\Programs\cursor\resources\...
C:\Users\pralhad\Desktop\file_folder_generator - Copy
C:\Users\pralhad\Desktop\file_folder_generator
C:\Program Files\NVIDIA Corporation\Nsight Compute 2025....
C:\Program Files\NVIDIA Corporation\NVIDIA App\NvDLISR
C:\Users\pralhad\AppData\Roaming\Python\Scripts
C:\Users\pralhad\Desktop\MyTools
C:\Users\pralhad\AppData\Local\Programs\Python\Python39...
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\

New Edit Browse... Delete Move Up Move Down Edit text...

OK Cancel

Select Python path

click to remove path

Click to save changes

Output: On Window

```
Microsoft Windows [Version 10.0.22631.5699]  
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\pralhad>python3
```

```
'python3' is not recognized as an internal or external command,  
operable program or batch file.
```

```
C:\Users\pralhad>|
```

→ System cannot find "python" Command

Why Is the PATH Variable Important?

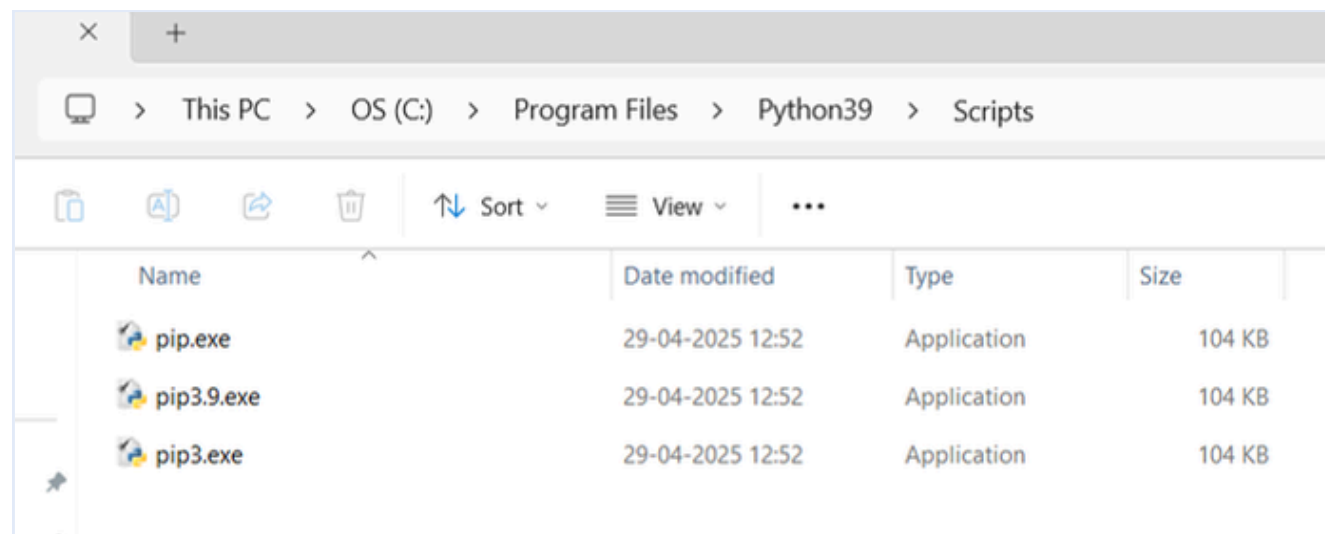
Running Python **Without** Specifying the **Full Path** Using **PATH**

- Let's say you want to run python on cmd

Location of Python Installation

Window

C:\Program Files\Python39\python.exe

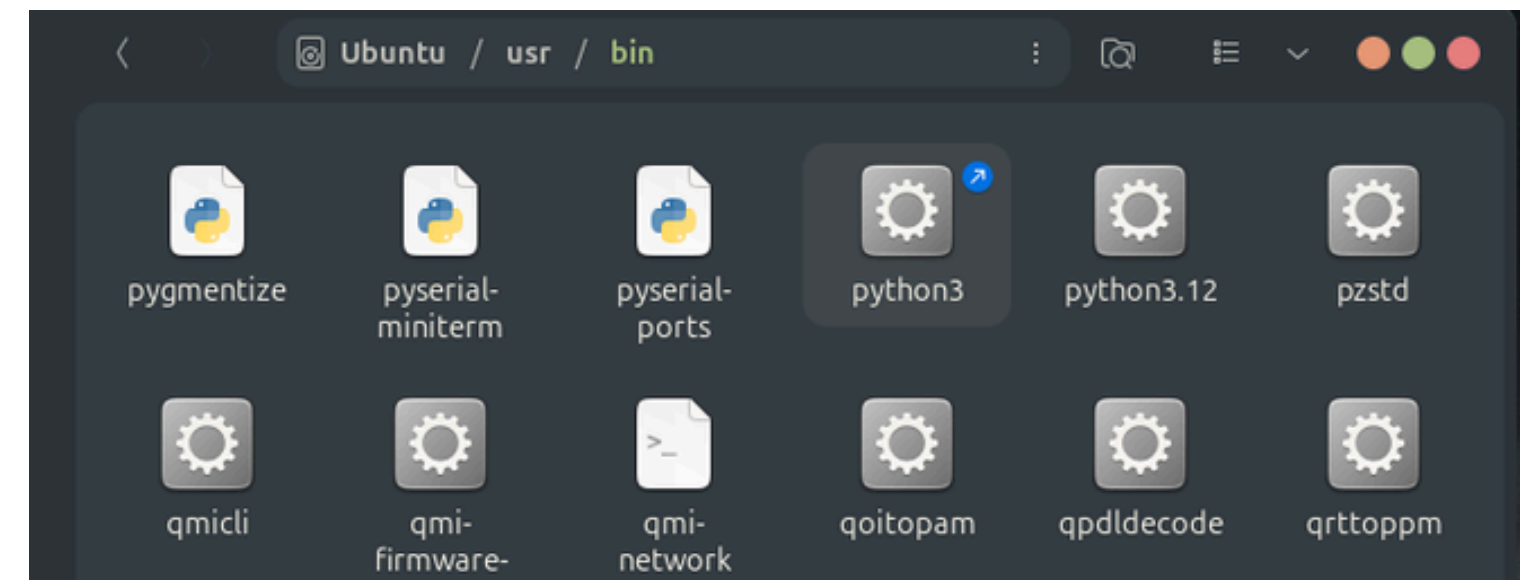


Terminal

```
C:\Users\pralhad>python
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42)
Type "help", "copyright", "credits" or "license" for more in
>>> |
```

MacOS/Linux

/usr/local/bin/python3



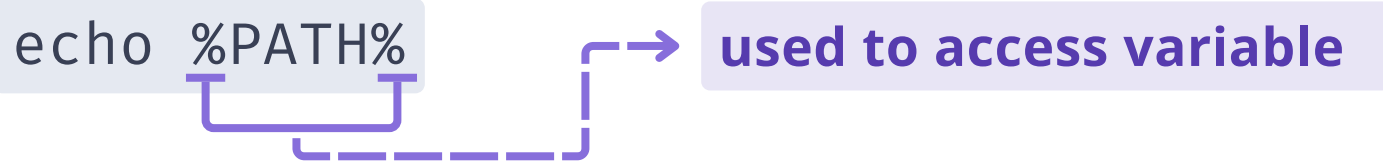
Terminal

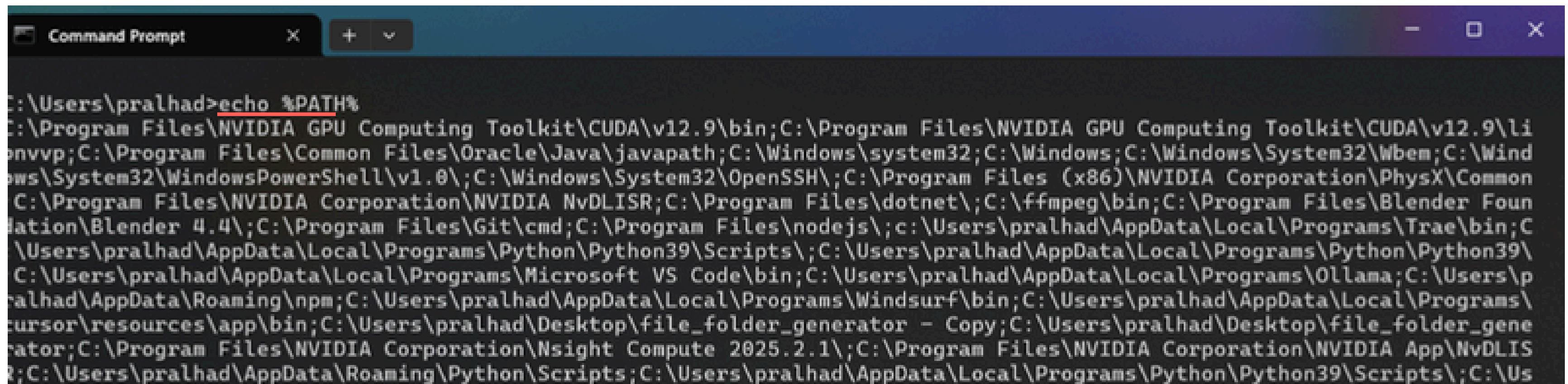
```
verlic@ubuntu: ~
verlic@ubuntu:~$ python3
Python 3.12.3 (main, Jun 18 2025, 17:59:45) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

This way the system knows where to run the command from.

Viewing the PATH Variable in the Terminal

On Windows (Command Prompt):

- Open CMD and Type → `echo %PATH%`  used to access variable



```
Command Prompt
C:\Users\pralhad>echo %PATH%
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9\bin;C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9\li
onvvp;C:\Program Files\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Wind
ows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common
;C:\Program Files\NVIDIA Corporation\NVIDIA NvDLISR;C:\Program Files\dotnet\;C:\ffmpeg\bin;C:\Program Files\Blender Foun
dation\Blender 4.4\;C:\Program Files\Git\cmd;C:\Program Files\nodejs\;c:\Users\pralhad\AppData\Local\Programs\Trae\bin;C
:\Users\pralhad\AppData\Local\Programs\Python\Python39\Scripts\;C:\Users\pralhad\AppData\Local\Programs\Python\Python39\
;C:\Users\pralhad\AppData\Local\Programs\Microsoft VS Code\bin;C:\Users\pralhad\AppData\Local\Programs\Ollama;C:\Users\p
ralhad\AppData\Roaming\npm;C:\Users\pralhad\AppData\Local\Programs\Windsurf\bin;C:\Users\pralhad\AppData\Local\Programs\
cursor\resources\app\bin;C:\Users\pralhad\Desktop\file_folder_generator - Copy;C:\Users\pralhad\Desktop\file_folder_gene
rator;C:\Program Files\NVIDIA Corporation\Nsight Compute 2025.2.1\;C:\Program Files\NVIDIA Corporation\NVIDIA App\NvDLIS
R;C:\Users\pralhad\AppData\Roaming\Python\Scripts;C:\Users\pralhad\AppData\Local\Programs\Python\Python39\Scripts\;C:\Us
```

On Linux/macOS

- Type → `echo $PATH`  symbol is used in Unix-like systems to reference variables.

Modifying the PATH Variable

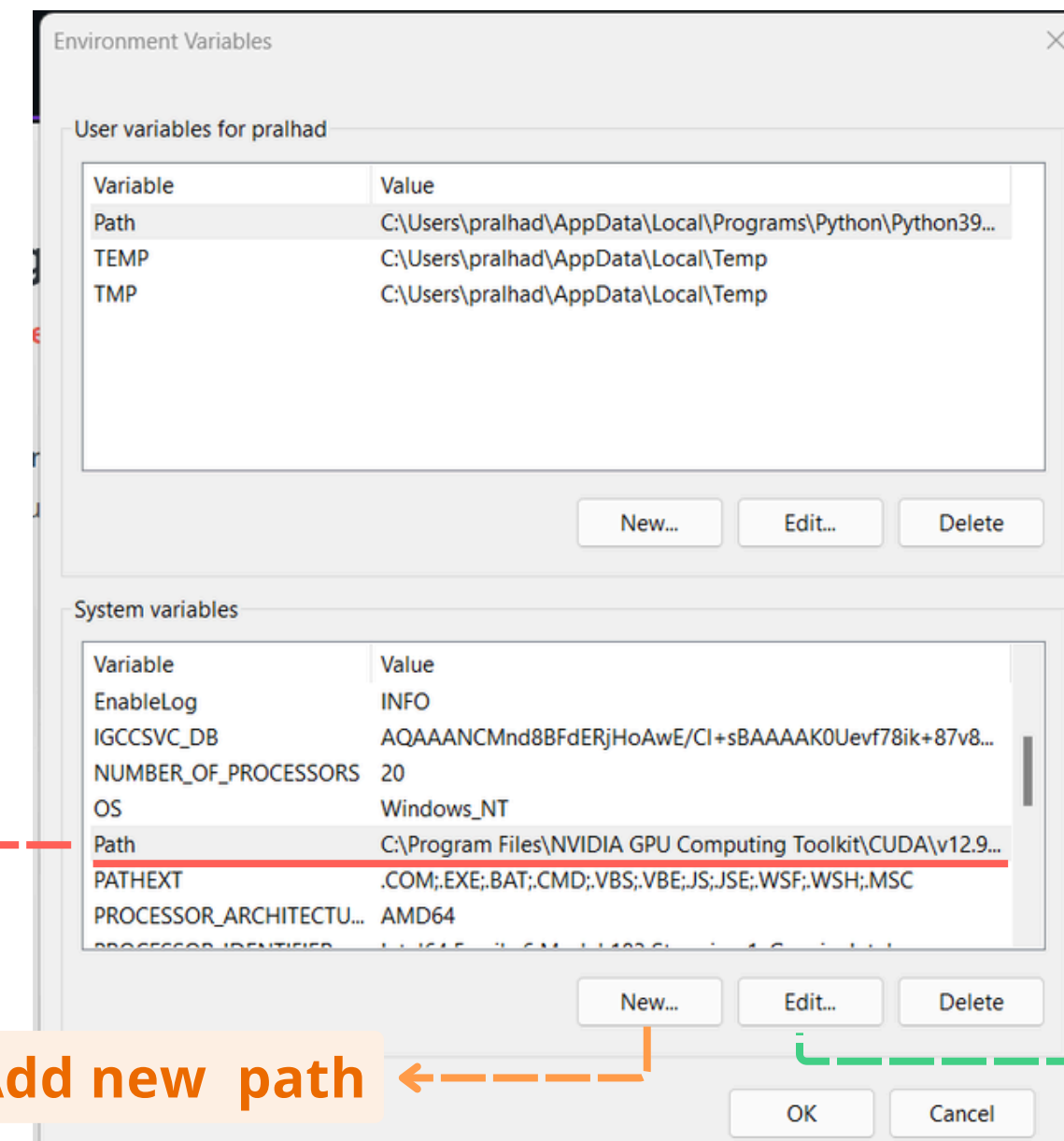
You can add **new** directories to the **PATH** variable to include **additional locations** for **executables**.

On Windows:

- Lets say we have a **folder** name **MyTools** in Desktop
- with **python** file having a **print** statement

A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows a folder named 'MYTOOLS' containing a file 'tool.py'. The main editor window shows the code in 'tool.py' as `1 print("Hello from Path")`. The file explorer at the top shows 'MyTools'.

- Open System Properties → **Environment Variables**.
- Find **PATH** under **System Variables**.

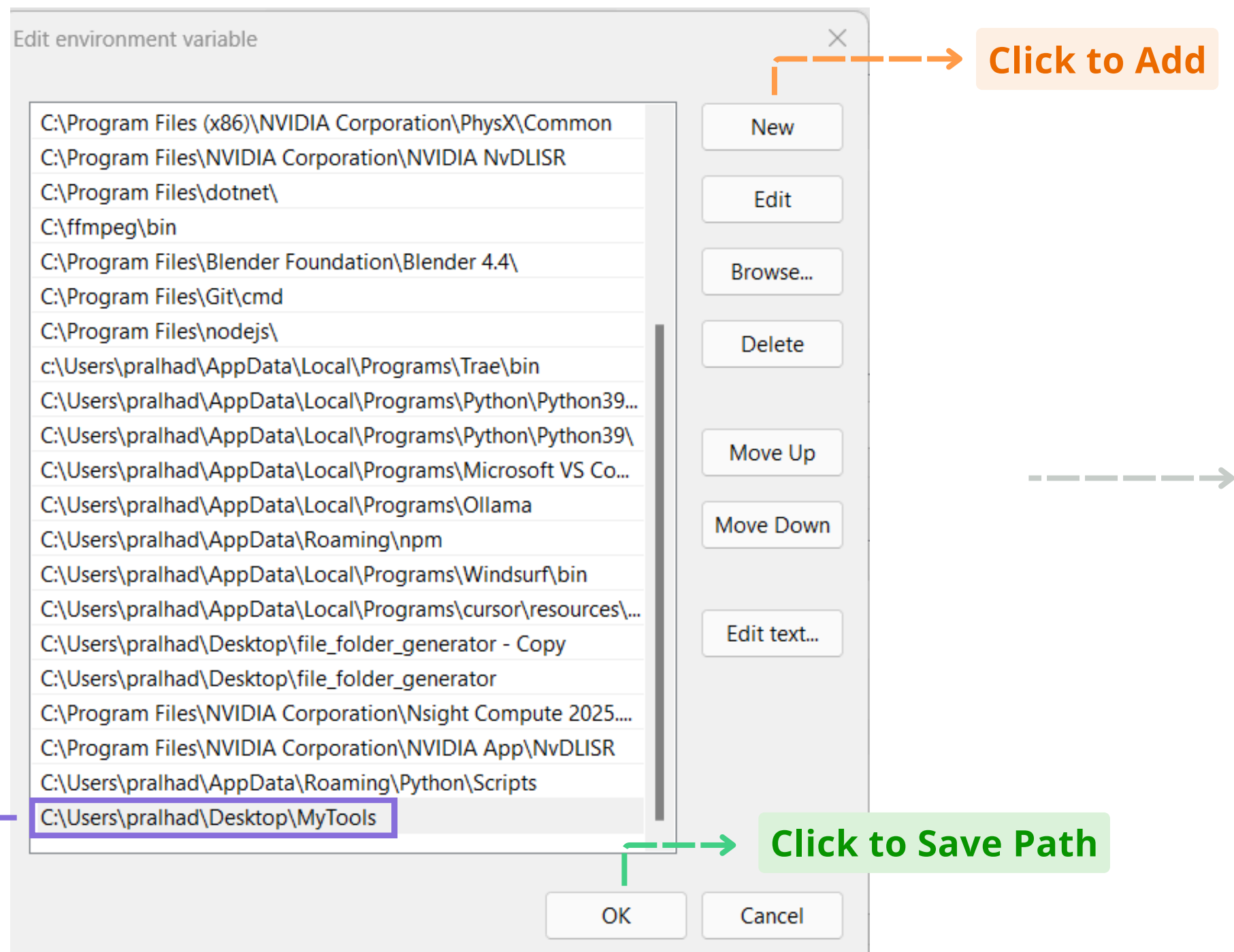


Select to Add or edit

Click to Add new path

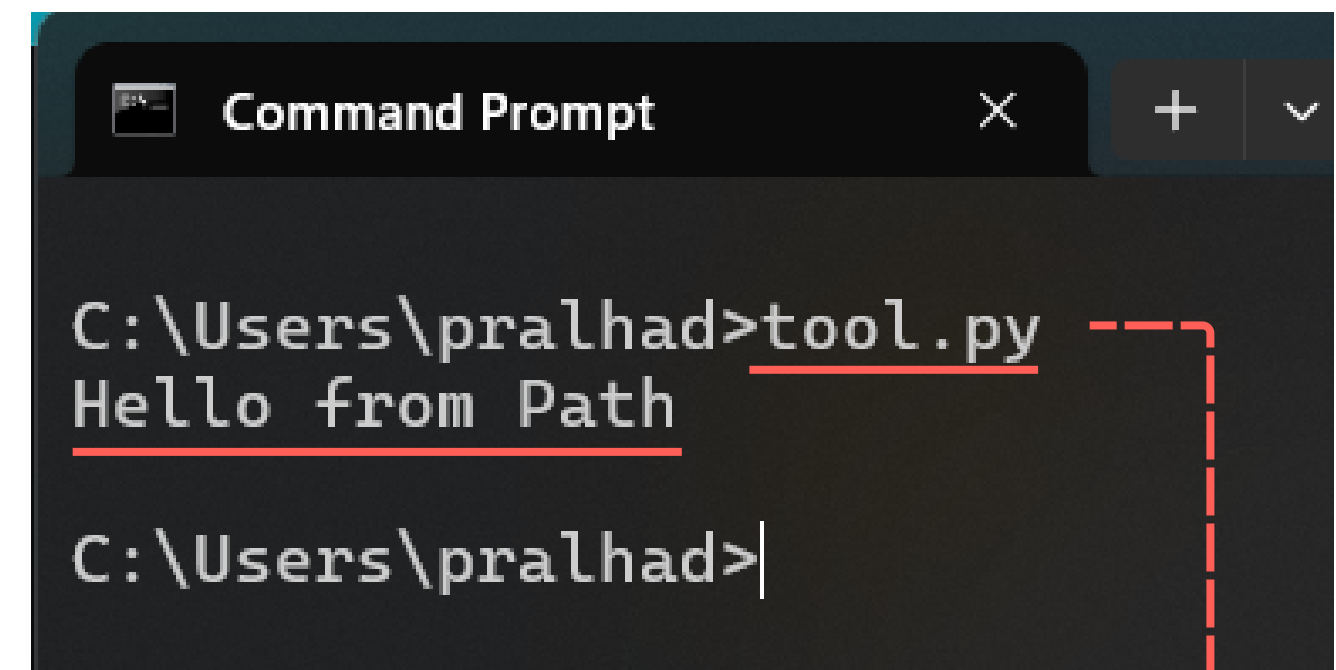
Click to edit

- Copy Folder Path and add it in Path Variable
- **Folder Path** → `C:\Users\pralhad\Desktop\MyTools`



Paste the path copied

- Open **CMD** and Try to run file
- without being in **same directory**



On Linux/macOS:

✓ Temporarily Add a Directory

→ `export PATH=$PATH:$HOME/tools/bin`

This change lasts only until you **close** the terminal.

✓ Make It Permanent

- Add the **export line** to your shell **config file**:

For Bash: → `nano ~/.bashrc`

For Zsh (macOS default): → `nano ~/.zshrc`

- Add this line at the end of the file

→ `export PATH=$PATH:/path/to/your/directory`

- Then apply the change:

→ `source ~/.bashrc # or source ~/.zshrc`

Common Issues While Getting PATH

1. Command Not Found Error

- Happens when the **command** you're trying to **run isn't** in any of the **directories listed in PATH**.
- **Example:** if Python **isn't installed** or its folder isn't in **PATH**, typing **python** gives an **error**.

2. Duplicate Directories

- Some directories may appear **multiple times** in **PATH**.
- Doesn't usually **break anything** but can **slow down** command lookups **slightly**.

3. Incorrect Syntax

- On **Windows** → use **semicolons** ; between paths.
- On **Linux/macOS** → use **colons** : between paths.

```
C:\Python39\;C:\Windows\System32\
```

```
/usr/local/bin:/usr/bin:/bin
```

Mistakes here can make the entire variable invalid.

Working with ENV Variables in Python

- Checking the Type of PATH
- Splitting the PATH into Individual Directories
- Setting Environment Variable
- Viewing Environment Variables
- Removing Environment Variables

Checking the Type of PATH

- When we retrieve the **PATH environment variable** using Python.
- it's important to know **what kind of data** we're working with.

```
import os
path = os.getenv("PATH")
print(type(path))
```

returns the value as a string

return data type

Output: <class 'str'>

Splitting the PATH Into Individual Directories

- **PATH** contains **many folders**, separated by:
 - **;** on Windows
 - **:** on Linux/macOS
- When you run a command, the system searches these **in order**
- Splitting lets us **see each folder clearly**, check for issues, or debug

Splitting the PATH into Separate Directories

- **On Windows:**

```
import os
```

```
path = os.getenv("PATH")  
directories = path.split(";")
```

```
for d in directories:  
    print(d)
```

Split by semicolon on Windows



```
# Output:  
c:/Users/HP/Desktop/dev/main.py  
C:\Windows\system32  
C:\Windows  
C:\Windows\System32\Wbem  
C:\Windows\System32\WindowsPowerShell\v1.0\  
C:\Windows\System32\OpenSSH\  
C:\Program Files\Git\cmd  
C:\Program Files\GitHub\CLI\  
D:\Python3_9_7\Scripts\  
D:\Python3_9_7\  
D:\python_3_7_9\Scripts\  
D:\python_3_7_9\  
C:\Users\HP\AppData\Local\Programs\Microsoft VS Code\bin
```

- On **Linux/macOS**, use **:** instead of **;**

Why This Helps

- Catch missing or invalid paths
- Spot duplicate entries
- See if the directory you need (e.g. **Python, Maya**) is included
- Understand search order — the **OS checks** folders top to bottom

```
# Output:  
c:/Users/HP/Desktop/dev/main.py  
C:\Windows\system32  
C:\Windows  
C:\Windows\System32\Wbem  
C:\Windows\System32\WindowsPowerShell\v1.0\  
C:\Windows\System32\OpenSSH\  
C:\Program Files\Git\cmd  
C:\Program Files\GitHub CLI\  
D:\Python3_9_7\Scripts\  
D:\Python3_9_7\  
D:\python_3_7_9\Scripts\  
D:\python_3_7_9\  
C:\Users\HP\AppData\Local\Programs\Microsoft VS Code\bin
```

Setting Environment Variable

- **Environment variables** are essential for controlling how **tools** behave
- In **VFX pipelines** where you may want to **set plugin paths, tool configs, or overrides dynamically**.

Temporarily Setting an Environment Variable - Window

We can **temporarily** set **environment variables**

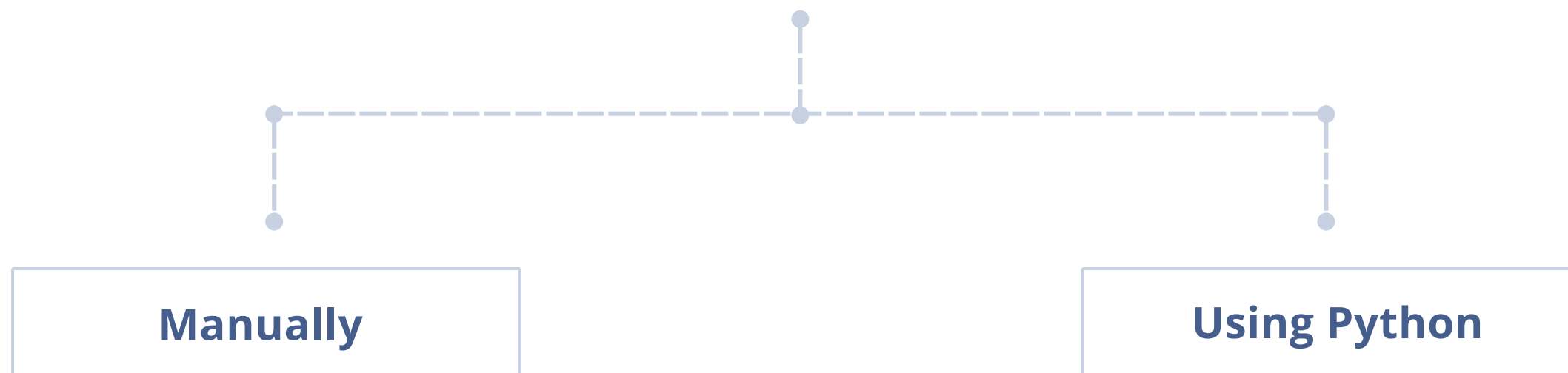
```
import os
os.environ['MAYA_PLUG_IN_PATH'] = 'Z:\\StudioTools\\CustomPlugin'
print(f"Updated Maya Plugin Path: {os.environ['MAYA_PLUG_IN_PATH']}")
# Updated Maya Plugin Path: Z:\\StudioTools\\CustomPlugin
```

used to set new Environment Variable

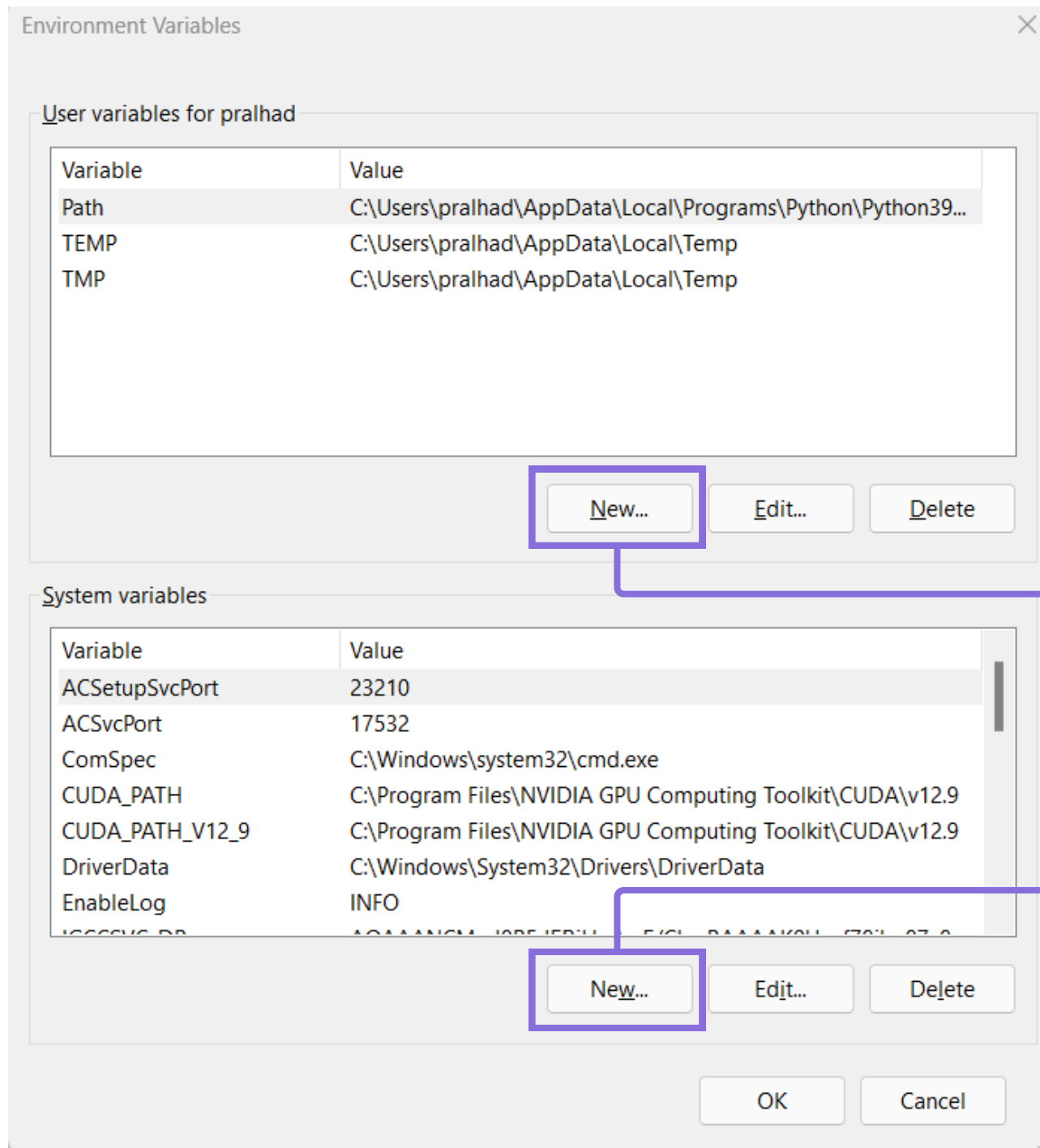
Work only current Python script session.

Great for **testing, debugging**, or **launching temporary custom tools**.

Permanently Setting an Environment Variable Window



Using GUI



- click **start** icon search **env** and **open it**
- Click **Environment Variables**
- **Add** ariable in **System** or **User variables**

Click to Add new variable

New User Variable

Variable name: MAYA_PLUG_IN_PATH

Variable value: Z:\StudioTools\CustomPlugin

Browse Directory... Browse File... OK Cancel

Key Path

Click to Save

Environment Variables

User variables for pralhad

Variable	Value
MAYA_PLUG_IN_PATH	Z:\StudioTools\CustomPlugin
Path	C:\Users\pralhad\AppData\Local\Programs\Python\Python39...
TEMP	C:\Users\pralhad\AppData\Local\Temp
TMP	C:\Users\pralhad\AppData\Local\Temp

New... Edit... Delete

Custom ENV Variable is set

Using Python — for Infrastructure / IT

Scenario:

- In a studio, **dozens of workstations** must have the **same environment variables**
- **e.g.**, `MAYA_PLUG_IN_PATH` `PYTHONPATH` `USD_ROOT`
- Doing this by hand on each machine is **slow** and **error-prone**.

Use a script because it's:

- **Consistent:** One script sets the **same values** on **every workstation**.
- **Repeatable & auditable:** You can run it anytime and keep a log of what was set.
- **Fast to roll out:** Push once; apply to **10–50+** machines via your deployment tool.

Disclaimer (very important):

- **Developers** → usually **don't** set system-wide env vars via Python. They read env vars for their tools.
- **Infrastructure/IT** → sets env vars **once per workstation** (or via deployment) so every user/tool is consistent.

- To **persist** a **new** variable across **terminal sessions**:

Windows Registry module

```
import winreg
import os
```

```
VARs = {
    'MAYA_PLUG_IN_PATH': r'Z:\StudioTools\SuperRenderTool',
    'PYTHONPATH':        r'Z:\StudioTools\PythonLibs',
    'USD_ROOT':           r'Z:\StudioAssets\USD',
}
```

Dictionary holding ENV variables

Only affect current user

env var name

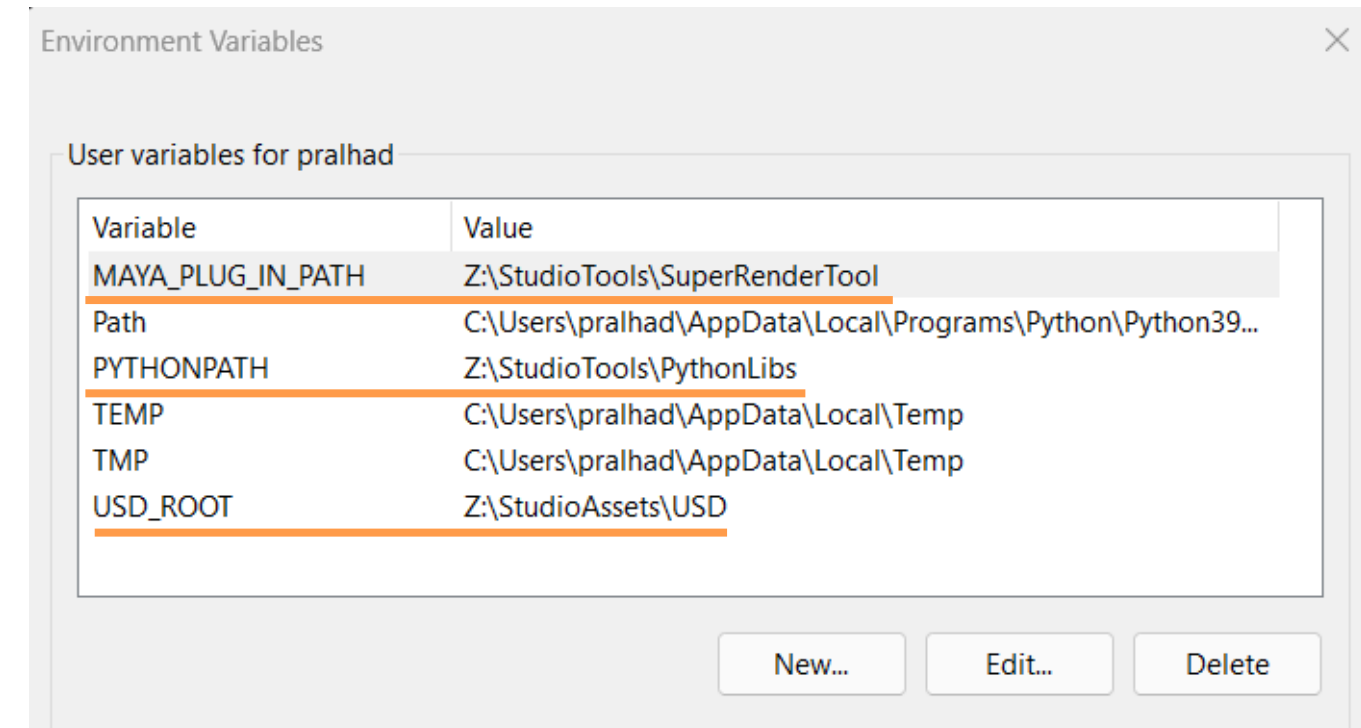
```
def set_user_env(vars_to_set):
    with winreg.OpenKey(winreg.HKEY_CURRENT_USER, r'Environment', 0, winreg.KEY_SET_VALUE) as key:
        for name, value in vars_to_set.items():
            winreg.SetValueEx(key, name, 0, winreg.REG_EXPAND_SZ, value)
            print(f"Set {name} = {value}")
```

Set keys

allows nested %VARs%

```
if __name__ == "__main__":
    set_user_env(VARS)
    print("\nDone. Restart the terminal (or sign out/in) to see changes.")
```

```
# Set MAYA_PLUG_IN_PATH = Z:\StudioTools\SuperRenderTool
# Set PYTHONPATH = Z:\StudioTools\PythonLibs
# Set USD_ROOT = Z:\StudioAssets\USD
# Done. Restart the terminal (or sign out/in) to see changes.
```



You must **restart your terminal** or **system** for the changes to take effect.

On Linux/macOS (Edit .bashrc or .zshrc)

- To persist the **variable** across **terminal sessions**, append it to your shell config file.

```
import os
with open(os.path.expanduser("~/ .bashrc"), "a") as bashrc:
    bashrc.write('\nexport MAYA_PLUG_IN_PATH=Z:\\\\StudioTools\\\\CustomPlugin\n')

print("Environment variable added to .bashrc. Run 'source ~/.bashrc' to apply changes.")
# Environment variable added to .bashrc. Run 'source ~/.bashrc' to apply changes.
```

Gets the full path to .bashrc

Append at bottom of file

set the ENV Variable

If you're using **Zsh** (macOS default), replace `~/ .bashrc` with `~/ .zshrc`.

Viewing Environment Variables

You can inspect the **environment variables** already set on your **system** using **Python**.

```
import os
```

Use to list both key and values

```
for key, value in os.environ.items():  
    print(f"{key}: {value}")
```

```
maya_plugin_path = os.environ.get('MAYA_PLUG_IN_PATH')  
print(f"Maya Plugin Path: {maya_plugin_path}")
```

```
# MAYA_PLUG_IN_PATH: Z:\StudioTools\SuperRenderTool
```

Output

```
ACSETUPSVCPORT: 23210  
ACSVCPORT: 17532  
ALLUSERSPROFILE: C:\ProgramData  
APPDATA: C:\Users\pralhad\AppData\Roaming  
CHROME_CRASHPAD_PIPE_NAME: \\.\pipe\crashpad_19744_TKIYSGNSTKFROBEL  
COMMONPROGRAMFILES: C:\Program Files\Common Files  
COMMONPROGRAMFILES(X86): C:\Program Files (x86)\Common Files  
COMMONPROGRAMW6432: C:\Program Files\Common Files  
COMPUTERNAME: LAPTOP-H0K45GFP  
COMSPEC: C:\Windows\system32\cmd.exe  
CUDA_PATH: C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9  
CUDA_PATH_V12_9: C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.9  
DRIVERDATA: C:\Windows\System32\Drivers\DriverData  
EFC_9456: 1  
ENABLELOG: INFO  
FPS_BROWSER_APP_PROFILE_STRING: Internet Explorer  
FPS_BROWSER_USER_PROFILE_STRING: Default  
HOMEDRIVE: C:  
HOMEPATH: \Users\pralhad  
LOCALAPPDATA: C:\Users\pralhad\AppData\Local  
LOGONSERVER: \\LAPTOP-H0K45GFP  
MAYA_PLUG_IN_PATH: Z:\StudioTools\CustomPlugin  
NUMBER_OF_PROCESSORS: 20  
ORIGINAL_XDG_CURRENT_DESKTOP: undefined  
OS: Windows_NT  
PATHEXT: .COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC;.CPL  
PROCESSOR_ARCHITECTURE: AMD64  
PROCESSOR_IDENTIFIER: Intel64 Family 6 Model 183 Stepping 1, GenuineIntel  
PROCESSOR_LEVEL: 6  
PROCESSOR_REVISION: b701  
PROGRAMDATA: C:\ProgramData  
PROGRAMFILES: C:\Program Files  
PROGRAMFILES(X86): C:\Program Files (x86)  
PROGRAMW6432: C:\Program Files
```

- Uses `.get()` to safely access a specific variable
- Returns `None` if the variable isn't set (avoids `KeyError`)

Removing Environment Variables

Temporarily Remove (Session Only)

```
import os
os.environ.pop('MAYA_PLUG_IN_PATH', None)
print("MAYA_PLUG_IN_PATH has been removed ")

# MAYA_PLUG_IN_PATH has been removed (temporarily).
```

Remove from the current script's ENV dictionary.

fallback value

This affects only the **running script** — it **doesn't** delete the variable from the **system**.

Permanently Remove on Windows

```
import winreg
with winreg.OpenKey(winreg.HKEY_CURRENT_USER, r'Environment', 0, winreg.KEY_SET_VALUE) as reg_key:
    winreg.DeleteValue(reg_key, 'MAYA_PLUG_IN_PATH')
print("MAYA_PLUG_IN_PATH removed from system environment variables.")

# MAYA_PLUG_IN_PATH removed from system environment variables.
```

Targets the current user's ENV settings

deletes the specified variable from the registry.

As with setting a **variable**, you'll need to **restart** the **terminal** for this **change to take effect**.

Summary

Task	Method	Persistence
• Set temporarily in Python	→ <code>os.environ['VAR'] = value</code>	→ Current session
• Set permanently (Windows)	→ <code>winreg.SetValueEx()</code>	→ System-wide
• Set permanently (Linux/macOS)	→ Add export line to <code>.bashrc</code>	→ System-wide
• View variable	→ <code>os.environ.get('VAR')</code>	→ Read access
• Remove temporarily	→ <code>os.environ.pop('VAR')</code>	→ Current session
• Remove permanently (Windows)	→ <code>winreg.DeleteValue()</code>	→ System-wide

Multiple Ways to get Environment

- Using `os.environ.get( )`
- Using `os.environ[ ]`
- Using `os.getenv( )`
- Which One Should We Use?

Using os.environ.get()

This is the most **common** and **safest way** to **access an environment variable**.

```
import os

maya_plugin_path = os.environ.get('MAYA_PLUG_IN_PATH', 'Default/Path')

print(f"Maya Plugin Path: {maya_plugin_path}")
# Maya Plugin Path: Z:\StudioTools\CustomPlugin
```

fallback default values

◆ Why Use It?

- Returns **None** if the variable **doesn't exist** — avoids crashing
- This is helpful when the variable is optional.

Using os.environ[]

This gives **direct dictionary-style access to the variable**.

```
import os

maya_plugin_path = os.environ['MAYA_PLUG_IN_PATH']

print(f"Maya Plugin Path: {maya_plugin_path}")
# Maya Plugin Path: Z:\StudioTools\CustomPlugin
```

use to access value of specified key

Warning:

- If the variable is **not set**, this will raise a **KeyError**

Best Practice for safety

If you're not sure the variable will be there, use a **try-except block**:

```
import os

try:
    maya_plugin_path = os.environ['NUKE_PATH']
    print(f"Maya Plugin Path: {maya_plugin_path}")
except KeyError:
    print("Error: MAYA_PLUG_IN_PATH environment variable is not set.")

# Error: MAYA_PLUG_IN_PATH environment variable is not set.
```

get ENV Value

Handle Key not found error

- Useful when we want to **handle error gracefully**

Using os.getenv()

This is a **shortcut** or **alias** for **os.environ.get()** — it works the same way but **uses** a **different function name**.

```
import os  
  
maya_plugin_path = os.getenv('MAYA_PLUG_IN_PATH')  
  
print(f"Maya Plugin Path: {maya_plugin_path}")  
# Maya Plugin Path: Z:\StudioTools\CustomPlugin
```

get value of given key

- Just like **.get()**, it returns **None** if the variable isn't found.

Optional Default Value

You can also set a default value:

```
maya_plugin_path = os.getenv('MAYA_PLUG_IN_PATH', 'Default/Path')
```

Great for readability and quick one-liners.

set default value

Which One Should We Use?

Use `os.environ.get()` or `os.getenv()`

- The environment variable **might not exist**
- You want to provide a **default fallback value**
- You want the script to continue running safely

```
path = os.environ.get('MY_VAR', 'default_value')
```

Default Fallback value ←

Best for **optional** or **configurable** settings
Example: Tool paths, **optional** overrides, **flags**

Use `os.environ[]`

- The environment variable **must be set**
- You want the script to **fail loudly** if it's missing
- You want to ensure required configuration is present

```
path = os.environ['MY_VAR'] # Raises KeyError if not set
```

Best for **critical variables**
e.g. license keys, API tokens

Comparison Table

Method	Purpose	Best For
• os.environ.get()	→ Returns None	→ When the variable is optional
• os.environ[]	→ Raises KeyError	→ When the variable must exist
• os.getenv()	→ Returns None	→ Cleaner alias for get(), behaves same

Using Environment Variables

- Connecting to a Database
- Loading JSON Config Files from a Common Folder
- Hardcoded Path vs Environment Variable
- Best Practices

Connecting to a Database

Imagine you are writing a script to connect to a **database**.

- The database requires a **username** and **password**.

Without Environment Variables

```
def connect_to_database():  
    db_user = "admin"  
    db_password = "password123"  
  
    print("Connecting to database with:")  
    print(f"Username: {db_user}")  
    print(f"Password: {db_password}")
```

```
connect_to_database()
```

```
# Connecting to database with:  
# Username: admin  
# Password: password123
```

Hardcoded Credentials

Why This Is a Problem

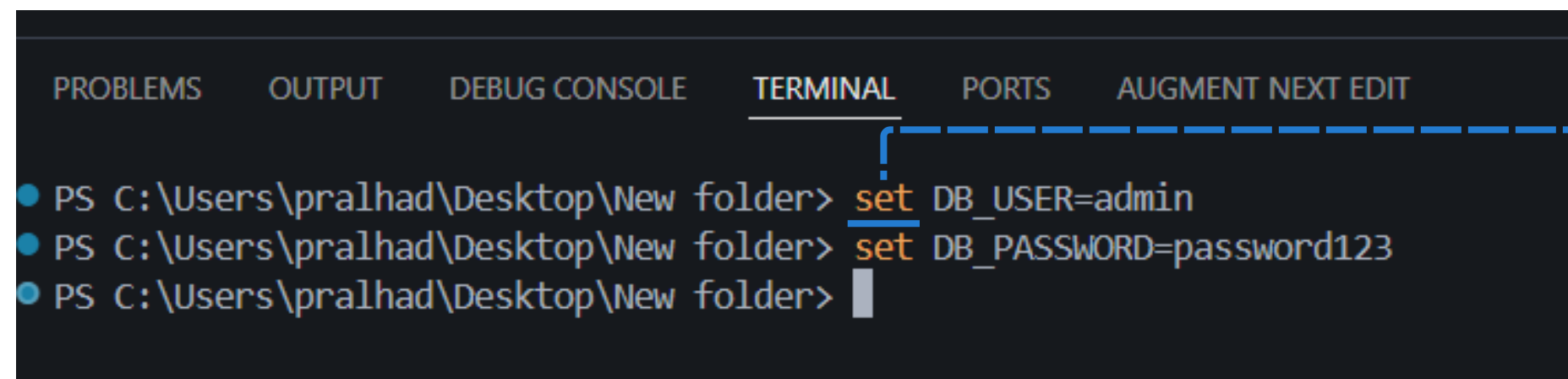
- **Security Risk** → Passwords are exposed in the code.
- **Code Updates Required** → Any change requires editing the script manually
- **Not Scalable**: → If reused across scripts, each one must be updated separately
- **Not Portable** → Sharing script with others exposes sensitive data.

Solution: With Environment Variables

Set Environment Variables

- Before running the **script**, define environment variables in same

On Windows (CMD):



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AUGMENT NEXT EDIT
PS C:\Users\pralhad\Desktop\New folder> set DB_USER=admin
PS C:\Users\pralhad\Desktop\New folder> set DB_PASSWORD=password123
PS C:\Users\pralhad\Desktop\New folder>
```

Used to set ENV Variable

On Linux/Mac (Terminal):

```
export DB_USER=admin
export DB_PASSWORD=password123
```

Note:

- Set **environment variables** in the same folder using **VS Code terminal**, then run your script in that **terminal session**.

Use Environment Variables in the Script

```
import os

def connect_to_database():
    db_user = os.environ.get('DB_USER')
    db_password = os.environ.get('DB_PASSWORD')

    if not db_user or not db_password:
        print("Error: Database credentials are not set in environment variables.")
        return

    print("Connecting to database with:")
    print(f"Username: {db_user}")
    print(f"Password: {'*' * len(db_password)}")

connect_to_database()
```

Get Key and Values using get method

Masked PASS For Security

Output:

```
C:\Users\pralhad\Desktop\New folder>python c.py
Connecting to database with:
Username: admin
Password: *****
```

Masked Password

Hardcoded vs Environment Variable

Aspect	Hardcoded	Environment Variable
• Security	→ Exposed in plain text	→ Credentials hidden in environment
• Flexibility	→ Hard to change	→ Easily update credentials globally
• Scalability	→ Needs updating in each script	→ One change affects all scripts

Key Takeaways

- **Without Environment Variables:** Sensitive data is exposed, hard to maintain.
- **With Environment Variables:** Secure, flexible, and easy to manage.
- Environment variables provide a **clean separation between code and configuration**.

Loading JSON Config Files from a Common Folder

- In real VFX pipelines, **configuration files** are **everywhere**: database settings, logging behavior, tool metadata,
- **Managing** them without **hardcoding paths** makes your **scripts portable, scalable**, and **easy to maintain**.

Scenario:

- We have **two Python tools**:
 - **tool1.py** → Reads DB config for asset management
 - **tool2.py** → Reads DB config for publishing scenes
- Both use the **same** config file: `/configs/database.json`

Sample database.json

```
{
  "host": "localhost",
  "port": 5432,
  "username": "admin",
  "password": "securepass",
  "database": "vfx_assets"
}
```

Without Environment Variables

◆ tool1.py

```
import os, json
```

Hardcoded relative path

```
def load_db_config():
```

```
    cur_path = os.path.dirname(__file__)
    config_path = os.path.join(cur_path, "configs", "database.json")
    with open(config_path, 'r') as file:
        return json.load(file)
```

```
def list_assets():
```

```
    cfg = load_db_config()
    print(f"Connecting to {cfg['database']} DB...")
    print("✅ Found 125 assets.")
```

```
if __name__ == "__main__":
    list_assets()
```

```
# Connecting to vfx_assets DB...
# ✅ Found 125 assets.
```

◆ tool2.py

```
import os, json, datetime
```

Hardcoded relative path

```
def load_db_config():
```

```
    cur_path = os.path.dirname(__file__)
    config_path = os.path.join(cur_path, "configs", "database.json")
    with open(config_path, 'r') as file:
        return json.load(file)
```

```
def publish_scene(scene_name):
```

```
    cfg = load_db_config()
    print(f"Connecting to {cfg['database']} ")
    print(f"Scene '{scene_name}' published at {datetime.datetime.now()}")
```

```
if __name__ == "__main__":
    publish_scene("episode01_scene05")
```

```
# Connecting to vfx_assets
# Scene 'episode01_scene05' published at 2025-08-09 18:11:15.652196
```


Problem: Each Tool Hardcodes Its Own DB Config Path

- Both tools point to their own copy of **database.json** (or hardcoded paths).
- Why this is not ideal:
 - **Duplication** → Same config repeated in **multiple places**.
 - **High maintenance** → Any DB change (host, user, password) requires **editing every tool**.
 - **Inconsistency risk** → Tools can drift out of sync if one file isn't updated.
 - **Not portable** → Paths differ across **machines/OS**.

Solution: With Environment Variables

- Create **one shared `database.json`**.
- Store its **path in a system-level environment variable (e.g., `DB_CONFIG_PATH`)**.
- All tools **read the path from the env var**, so they load the **same file**.

Set the Environment Variable

- **Windows (CMD):**

```
set DB_CONFIG_PATH=C:\Project\configs\database.json
```

- **Linux/Mac (Terminal):**

```
export DB_CONFIG_PATH=/home/user/project/configs/database.json
```


Update Both Tools to Use It

◆ tool1.py

Safely fetch ENV Variable

```
import os, json

def load_db_config():
    config_path = os.environ.get('DB_CONFIG_PATH')
    if not config_path:
        raise RuntimeError("DB_CONFIG_PATH environment variable is not set")
    with open(config_path, 'r') as file:
        return json.load(file)

def list_assets():
    cfg = load_db_config()
    print(f"Connecting to {cfg['database']} DB...")
    print("✅ Found 125 assets.")

if __name__ == "__main__":
    list_assets()

# Connecting to vfx_assets DB ...
# ✅ Found 125 assets.
```

◆ tool2.py

Safely fetch ENV Variable

```
import os, json, datetime

def load_db_config():
    config_path = os.environ.get('DB_CONFIG_PATH')
    if not config_path:
        raise RuntimeError("DB_CONFIG_PATH environment variable is not set")
    with open(config_path, 'r') as file:
        return json.load(file)

def publish_scene(scene_name):
    cfg = load_db_config()
    print(f"Connecting to {cfg['database']} DB...")
    print(f"✅ Scene '{scene_name}' published at {datetime.datetime.now()}")

if __name__ == "__main__":
    publish_scene("episode01_scene05")

# Connecting to vfx_assets DB...
# Scene 'episode01_scene05' published at 2025-08-09 18:11:15.652196
```

Handles missing env variable

Another Example Multiple JSON file

- The same ENV variable approach isn't limited to a **single config file** like **DB_CONFIG_PATH**.
 - We can **store a folder path** in one variable (e.g., **CONFIG_FOLDER**) and load multiple configs from it.
 - Or We can **set multiple environment variables**, one for each important config file, tool, or library path.

Folder structure

- Instead of **hardcoding the path to each file**,
- we'll use an **ENV variable** to point to the **/configs** folder.

```
/configs
├── database.json
├── logging.json
└── app.json
```

database.json

```
{
  "host": "localhost",
  "port": 5432,
  "username": "admin",
  "password": "securepass",
  "database": "vfx_assets"
}
```

logging.json

```
{
  "level": "DEBUG",
  "file": "logs/app.log",
  "format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
}
```

app.json

```
{
  "app_name": "MyVFXTool",
  "version": "1.0.0",
  "author": "Pipeline Team",
  "features": ["asset_import", "scene_validation", "auto_publish"]
}
```

Without Environment Variables

Script

```
import json
import os

def load_config(config_filename):

    config_folder = "C:/Users/pralhad/Desktop/New folder/configs"
    config_path = os.path.join(config_folder, config_filename)

    try:
        with open(config_path, 'r') as file:
            config = json.load(file)
            print(f"\nLoaded configuration from: {config_path}")
            print(json.dumps(config, indent=2))
    except FileNotFoundError:
        print(f"Error: Config file '{config_filename}' not found in {config_folder}")
    except json.JSONDecodeError:
        print(f"Error: Failed to parse JSON in {config_filename}")

if __name__ == "__main__":
    load_config('database.json')
    load_config('logging.json')
    load_config('app.json')
```

Load a JSON Config file from a hardcoded path

Output

```
Loaded configuration from: C:/Users/pralhad/Desktop/New folder/configs/database.json
{
  "host": "localhost",
  "port": 5432,
  "username": "admin",
  "password": "securepass",
  "database": "vfx_assets"
}

Loaded configuration from: C:/Users/pralhad/Desktop/New folder/configs/logging.json
{
  "level": "DEBUG",
  "file": "logs/app.log",
  "format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
}

Loaded configuration from: C:/Users/pralhad/Desktop/New folder/configs/app.json
{
  "app_name": "MyVFXTool",
  "version": "1.0.0",
  "author": "Pipeline Team",
  "features": [
    "asset_import",
    "scene_validation",
    "auto_publish"
  ]
}
```

❌ Problems with Hardcoded Paths

- Not portable across systems (Windows vs. Linux)
- Any path change requires editing the script
- Easy to break due to typos or rigid structure

With Environment Variables

Set Environment Variable

- On Windows (CMD)

```
set CONFIG_FOLDER=C:\Project\configs
```

- On Linux/Mac (Terminal):

```
export CONFIG_FOLDER=/home/user/project/configs
```


Script

```
import os, json

def load_config(config_filename):
    config_folder = os.environ.get('CONFIG_FOLDER')

    if not config_folder:
        print("❌ Error: CONFIG_FOLDER environment variable is not set.")
        return

    config_path = os.path.join(config_folder, config_filename)

    try:
        with open(config_path, 'r') as file:
            config = json.load(file)
            print(f"\n✅ Loaded configuration from: {config_path}")
            print(json.dumps(config, indent=2))
    except FileNotFoundError:
        print(f"❌ File not found: {config_filename}")
    except json.JSONDecodeError:
        print(f"❌ Invalid JSON in file: {config_filename}")

load_config('database.json')
load_config('logging.json')
```

Safely fetch CONFIG_FOLDER

Handles missing env variable

Builds the full file path dynamically

Handles bad JSON

Handles file not found errors

Output

```
✅ Loaded configuration from: C:\Users\pralhad\Desktop\New folder\configs\database.json
{
  "host": "localhost",
  "port": 5432,
  "username": "admin",
  "password": "securepass",
  "database": "vfx_assets"
}

✅ Loaded configuration from: C:\Users\pralhad\Desktop\New folder\configs\logging.json
{
  "level": "DEBUG",
  "file": "logs/app.log",
  "format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
}
```

Advantages of Using Environment Variables:

- **Flexibility:** Update the folder path without **modifying** the script.
- **Portability:** Works seamlessly across **Windows, macOS**, and **Linux**.
- **Scalability:** Add more configuration files without **script changes**.
- **Cleaner Code:** Removes **hardcoded** folder **paths** from **the script**.
- **Error Reduction:** Centralizes **configuration**, reducing **path-related errors**.

Hardcoded Path vs Environment Variable

Aspect	Hardcoded Path	Environment Variable & Relative Path
• Portability	→ ❌ Machine-specific	→ ✅ Platform-agnostic
• Flexibility	→ ❌ Hard to change globally	→ ✅ Centralized control via env variables
• Scalability	→ ❌ Requires manual updates	→ ✅ Easy to add more files dynamically
• Maintenance	→ ❌ Tedious to manage	→ ✅ Simplified path management
• Environment-Specific	→ ❌ OS and system-tied	→ ✅ Works across different environments

Best Practices

- Avoid hardcoding paths.
- Use `os.environ.get()` to fetch paths from environment variables.
- Use `os.path.join()` to build paths cleanly across platforms.
- Use **relative paths** when **dealing with files** inside your **project**.

When to Use Hardcoded Paths

- **Temporary Scripts:** For quick, one-off tasks or testing.
- **Personal Use:** If you're the only one using the project on your local machine.

Real-World Scenarios

- Managing Maya Plugins
- Shared Render Farm
- Centralized Houdini Assets
- Shared Python Library for All Tools

Managing Maya Plugins

🎯 Scenario:

- A VFX studio is producing an **animated feature**.
- 10 artists are using **Autodesk Maya**, and each one needs the **same custom tools and plugins**.

Problem Without Environment Variables

- **Artist A** installs the plugin at:
→ `Z:\StudioTools\SuperRenderTool`
- **Artist B** installs the plugin at:
→ `D:\StudioTools\SuperRenderTool`

Result:

- Each artist configures their own Maya plugin path manually
- Leads to **inconsistencies, missing tools**, and **time wasted debugging**

The Solution: Use an Environment Variable

Define a common variable:

→ MAYA_PLUG_IN_PATH

- Point it to a **shared network drive** that stores the plugin

Set it per OS:

- On Windows:

```
set MAYA_PLUG_IN_PATH=Z:\StudioTools\SuperRenderTool
```

- On Linux/Mac:

```
export MAYA_PLUG_IN_PATH=/mnt/StudioTools/SuperRenderTool
```

- Maya reads this path **automatically** at **startup**.

Shared Render Farm

Imagine a render farm where hundreds of machines need to **render frames** using tools like **Maya** and **Arnold** Renderer.

- Without environment variables, → need to **manually configure** every render node.
- With environment variables → one **central configuration file** can ensure all nodes are set up correctly.

Configuration :

```
set ARNOLD_SHADER_PATH=Z:\StudioTools\ArnoldShaders
```

- Now, every node references the same **shader** library,
- ensuring **consistent renders** across all frames

Centralized Houdini Assets

🎯 Scenario:

- A studio uses **Houdini** for **effects work**.
- Artists need access to a shared set of custom **Houdini Digital Assets (HDAs)** and **shelf tools**.

Problem Without Environment Variables

- Each artist keeps their own local copy of the HDAs in different folders:

```
C:\Users\Alice\HoudiniTools  
D:\FX\CustomHDAs
```

Result:

- Some artists use **outdated tools**, leading to version **mismatches** and **inconsistent results**.

Set a common environment variable:

- Windows:

```
set HOUDINI_PATH=Z:\StudioTools\HoudiniAssets
```

- Linux/Mac:

```
export HOUDINI_PATH=/mnt/StudioTools/HoudiniAssets
```

- Now Houdini automatically loads the **latest shared tools** for **everyone**.
- 

Shared Python Library for All Tools

🎯 Scenario:

- **Multiple** Python-based tools in the studio share the same **helper modules**
 - e.g., database connectors, asset tracking APIs

Problem Without Environment Variables

- Each tool keeps its **own copy** of the library in its project folder.

Result:

- Bug fixes require updating every tool's copy, which is **slow** and **error-prone**.

Solution:

- **Windows:**

```
set PYTHONPATH=Z:\StudioTools\PythonLibs
```

- **Linux/Mac:**

```
export PYTHONPATH=/mnt/StudioTools/PythonLibs
```

- Now all Python scripts automatically import the latest library without local copies.

Environment Variables in the VFX Industry

- Why Should Developers Care About ENV Variables?
- Why Are Environment Variables Valuable?
- Pro Tip for VFX Developers

Why Should Developers Care About ENV Variables?

1: Consistency Across Machines

- All artists and TDs use the **same paths, tools, and plugins**
- Reduces setup **errors** and **misconfigurations**

Example:

```
set MAYA_PLUG_IN_PATH=Z:\StudioTools\SuperRenderTool
```

- Everyone loads the **same plugin** regardless of their **local setup**.

2: Simplified Software Deployment (Scalability)

- No need to **install plugins** manually on **every machine**
- Just update the tool in the **central network location**

Example:

```
set HOUDINI_PATH=Z:\StudioTools\HoudiniAssets
```

- Houdini automatically picks up the latest version across the team.

3: Custom Paths for External Tools

Let's say your **Python tool depends** on:

- **Maya** for scripting
- **OpenImageIO** for image processing
- **USD** for scene description

Without environment variables, your script has to **guess** or **hardcode** these paths. That's **brittle** and **error-prone**.

With environment variables:

```
maya_path = os.environ.get('MAYA_PLUG_IN_PATH')  
usd_root = os.environ.get('USD_ROOT')
```

Result: Your tools **automatically adapt** to each **workstation's configuration**.

4: Automation and Scripting

- **Scripts** can dynamically **adapt** to studio setups using **env variables**
- Makes them **portable** across **users**, **machines**, and **projects**

```
import os
plugin_path = os.environ.get('MAYA_PLUG_IN_PATH')
print(f>Loading plugins from: {plugin_path}")
```

5: Cross-Platform Compatibility

- Studios often use a mix of:
 - **Windows** (for artists)
 - **Linux** (for render nodes/TDs)
 - **macOS** (for devs)
- Environment variables provide a **unified interface** for all OSes.

Linux:

```
export USD_ROOT=/mnt/StudioAssets/USD
```

OR

Windows:

```
set USD_ROOT=Z:\StudioAssets\USD
```

- **Same logic, different syntax** — paths remain consistent across platforms

6: Flexible Configuration for Multiple Environments

- In **software development**, we often work in stages:
 - **dev** — with verbose logging
 - **test** — with sandboxed data
 - **prod** — with real files, fewer logs
- Instead of **changing** code **every time**, We just **set an environment variable**: `set APP_ENV=production`

Then in Python:

```
import os

env = os.getenv('APP_ENV', 'development')
if env == 'production':
    log_level = 'ERROR'
    print(log_level)
else:
    log_level = 'DEBUG'
    print(log_level)

# ERROR
```

Result: One codebase, many behaviors — based on **environment**.

Key Takeaways:

- **Consistency:** Every workstation **behaves** the **same way**.
 - **Scalability:** **Easy** to **add** or update tools across the **studio**.
 - **Reduced Errors:** Eliminate manual **misconfigurations** using **dependency** management.
 - **Automation:** Smooth integration with **pipeline** scripts.
 - **Cross-Platform Flexibility:** Uniform setups across **Windows, Linux,** and **macOS**.
- 

Why Are Environment Variables Valuable?

GamutX

**Separating Application Code
From Configurations**

**Simplifying Application
Configuration**

**Managing Secrets
& Credentials**

**Promoting
Consistency**

Pro Tip for VFX Developers

- Always Document Key Environment Variables
- To ensure **clarity**, team **consistency**, and **easy** onboarding,
- maintain a central reference of **critical** environment variables in studio's technical **documentation** or **internal** wiki.

Examples

Variable

Purpose

- | | |
|--------------------------|--|
| • Command Palette | → Tells Maya where to find custom plugins or tools |
| • Settings Menu | → Used by Python to locate shared scripts and libraries |
| • Settings Menu | → Path to Universal Scene Description (USD) core libraries |
| • Settings Menu | → Location of shared Houdini tools and digital assets |
| • Settings Menu | → Shared Arnold shader library for consistent rendering |

Note :

Environment variable names are **conventionally written** in **uppercase letters**.

- This is a **widely accepted standard** in programming.
- It helps distinguish **environment-level settings** from regular variables in your code.

Examples

- **PYTHONPATH** → tells Python where to look for modules
- **APP_ENV** → defines the current environment (e.g., development, production)
- **HOUDINI_PATH** → used to load plugins or assets in Houdini

Set Plugin Path for a Render Engine

Simulate setting a render engine plugin path using an environment variable.

```
import os

# Set a temporary render engine plugin path
os.environ['RENDER_PLUGIN_PATH'] = 'Z:/StudioTools/RenderEnginePlugins'

# Retrieve and display the value
plugin_path = os.getenv('RENDER_PLUGIN_PATH')
print(f"Render Plugin Path: {plugin_path}")
```

Verify Texture Library Directory

- Create a directory Z:/StudioTools/Textures (or equivalent on your system) to simulate the texture library path.
- Check if the directory for a texture library specified by an environment variable exists.

```
import os

# Set a temporary environment variable for texture library
os.environ['TEXTURE_LIBRARY_PATH'] = 'Z:/StudioTools/Textures'

# Verify directory existence
texture_path = os.getenv('TEXTURE_LIBRARY_PATH')

if texture_path and os.path.exists(texture_path):
    print(f"The Texture Library is available at: {texture_path}")
else:
    print("Texture Library path is not set or does not exist.")
```

List Available Scripts

- Create a directory Z:/StudioTools/NukeScripts and add some dummy script files e.g., script1.py, script2.py)
- Use an environment variable to point to a directory containing scripts for a compositing tool like Nuke.

```
import os

# Set a temporary path for Nuke scripts
os.environ['NUKE_SCRIPT_PATH'] = 'Z:/StudioTools/NukeScripts'

# List all scripts in the directory
nuke_scripts_path = os.getenv('NUKE_SCRIPT_PATH')

if nuke_scripts_path and os.path.isdir(nuke_scripts_path):
    print("Available Nuke Scripts:")
    for script in os.listdir(nuke_scripts_path):
        print(f"- {script}")
else:
    print("Nuke script directory not found.")
```

Add Custom Shader Path

- Create a directory Z:/StudioTools/ArnoldShaders to simulate a shader path.
- Simulate adding a custom shader path for Arnold Renderer using an environment variable.

```
import os

# Set and retrieve Arnold shader path
os.environ['ARNOLD_SHADER_PATH'] = 'Z:/StudioTools/ArnoldShaders'
shader_path = os.getenv('ARNOLD_SHADER_PATH')

if shader_path:
    print(f"Arnold Shader Path set to: {shader_path}")
else:
    print("Arnold Shader Path not defined.")
```


Set Multiple Paths

- Create directories
 - Z:/StudioTools/Assets/Models, Z:/StudioTools/Assets/Textures, and Z:/StudioTools/Assets/Rigs.
- Simulate setting multiple paths for different asset categories (models, textures, rigs) using environment variables.

```
import os

# Set environment variables for asset paths
os.environ['MODEL_PATH'] = 'Z:/StudioTools/Assets/Models'
os.environ['TEXTURE_PATH'] = 'Z:/StudioTools/Assets/Textures'
os.environ['RIG_PATH'] = 'Z:/StudioTools/Assets/Rigs'

# Print all paths
for asset_type in ['MODEL_PATH', 'TEXTURE_PATH', 'RIG_PATH']:
    path = os.getenv(asset_type)
    print(f"{asset_type}: {path if path else 'Not Set'}")
```

Debug Missing Plugin Path

- Create a directory Z:/StudioTools/MayaPlugins.
- Create a script to debug missing plugin paths for software like Maya.

```
import os

# Check for a plugin path environment variable
plugin_path = os.getenv('MAYA_PLUGIN_PATH')

if plugin_path:
    if os.path.isdir(plugin_path):
        print(f"Maya Plugin Path is set correctly: {plugin_path}")
    else:
        print(f"Error: Maya Plugin Path directory does not exist: {plugin_path}")
else:
    print("Error: Maya Plugin Path is not set.")
```

Thank You

